Tema 1

Iniciación a la programación

Programación

Tech Hub <FP>

euroformac

ÍNDICE

Objetivos de la unidad 4
1. Introducción 5
2. Datos algoritmos y programas 6
2.1. Datos 6
2.1.1 Tipos de datos: 6
2.2. Algoritmos 15
2.2.1 Diagramas de flujo 17
2.2.2 Pseudocódigos 18
2.3. Ejemplos de diferentes tipos de algoritmos 20
2.4. Algoritmos de búsqueda 21
2.5 Algoritmos para ordenar 23
2.6. Algoritmos voraces 26
2.7. Programación dinámica 27
2.8. Algoritmos probabilísticos 28
2.9 Ejemplo sencillo de creación de un algoritmo: 28
2.10. Programas 29
3. Paradigmas de programación 32
3.1. Programación imperativa 33
3.2. Programación declarativa 33
4. Lenguajes de programación 37
4.1. Máquinas y lenguajes de bajo nivel 38
4.2. Lenguaje de nivel intermedio 39
4.3. Lenguajes de alto nivel y de muy alto nivel 39
4.4. Ingeniería del Software 42
5. Herramientas y entornos para el desarrollo de programas 45
5.1 Entornos de desarrollo 45
5.2 Herramientas de control de versiones 46
5.3 Pruebas de software 50
5.3.1 Pruebas funcionales 51

Tech Hub <FP> euroformac

5.3.2 Las pruebas no funcionales incluyen 52
5.4 Documentación de los programas 53
6. Errores y calidad de los programas 55
6.1 Errores 55
6.2 Calidad del software 58
6.3 Estándares de calidad del software 60
6.4 Conclusiones 66
7. Resumen 67
Glosario 68
Bibliografía 69

Tech Hub <FP> euroformac

Objetivos de la unidad

Este curso de iniciación a la programación debe tener como objetivo principal introducir a los estudiantes en los conceptos fundamentales y las habilidades básicas necesarias para programar. Aquí hay algunos objetivos que debemos abordar:

- Aprender los conceptos básicos de la programación, como variables, tipos de datos, estructuras de control (como bucles y condicionales) y funciones. Con estos conceptos podremos comprender cómo se estructura un programa y cómo se ejecutan las instrucciones.
- Conocer un lenguaje de programación específico como Python o Java. Nos proporcionará una introducción a la sintaxis y las características básicas del lenguaje, permitiéndonos comenzar a escribir código y construir programas simples.
- La programación implica la capacidad de analizar problemas y encontrar soluciones eficientes. Un objetivo importante el módulo será desarrollar habilidades de resolución de problemas, enseñándonos a descomponer un problema en partes más pequeñas, identificar patrones y aplicando algoritmos para llegar a soluciones.
- Se fomentará el desarrollo de habilidades de pensamiento lógico, como la capacidad de razonar y seguir secuencias lógicas, identificar patrones y encontrar soluciones basadas en reglas y condiciones.
- Los algoritmos son pasos precisos y ordenados para resolver un problema. El módulo nos enseñará cómo diseñar algoritmos efectivos, cómo traducir los pasos en instrucciones de programación y cómo evaluar la eficiencia de los algoritmos.
- Se incluirán ejercicios prácticos para que podamos aplicar lo que hemos aprendido. Estas actividades nos permitirán practicar la escritura de código, experimentar con diferentes soluciones y desarrollar su capacidad para resolver problemas reales.
- Fomentar la creatividad, ya que la programación también es una forma de creatividad. El módulo nos animará a pensar de manera creativa y a buscar soluciones innovadoras. Incluirá proyectos donde podamos aplicar nuestra creatividad para desarrollar programas o juegos únicos.

Tech Hub <FP> euroformac

¿Cuál es la utilidad de la programación?

La programación informática tiene una amplia gama de utilidades y desempeña un papel fundamental en muchos aspectos de nuestras vidas en la actualidad. Es una habilidad fundamental en la era digital, ya que permite el desarrollo de software, la automatización de tareas, el análisis de datos, la creación de sitios web y aplicaciones, la conectividad de dispositivos y la seguridad informática.

1. Introducción

En este emocionante viaje, nos sumergiremos en el fascinante mundo de la programación y aprenderemos los fundamentos esenciales para convertirnos en programadores competentes. A lo largo de este curso, adquiriremos las habilidades y el conocimiento necesarios para crear nuestros propios programas y desarrollar soluciones innovadoras.

Nuestro objetivo principal es brindarte una base sólida en los conceptos fundamentales de la programación. Comenzaremos desde cero, sin suponer ningún conocimiento previo en el tema, y avanzaremos gradualmente para que podamos asimilar los conceptos de manera clara y concisa.

Aprenderemos cómo funcionan los lenguajes de programación y cómo comunicarnos con las computadoras a través del código. Descubriremos los bloques de construcción esenciales, como variables, tipos de datos, estructuras de control y funciones, y aprenderemos cómo combinarlos para crear programas simples pero efectivos.

Además, exploraremos el poder del pensamiento lógico y la resolución de problemas. Aprenderemos a analizar desafíos, descomponerlos en partes más pequeñas y aplicar algoritmos para encontrar soluciones eficientes. Desarrollaremos habilidades clave en el diseño de algoritmos y la planificación de programas.

2. Datos algoritmos y programas

2.1. Datos

Definición

Un tipo de dato en programación es una forma de clasificar los datos según sus propiedades. Puede tomar la forma de una cadena, texto, número, etc.

Todos los lenguajes de programación de computadoras, incluidos C, C++, Java y Python otorgan un gran valor a los tipos de datos. Estos deben asignarse y aplicarse correctamente cuando los programadores crean software de computadora, ya sea de escritorio o basado en la web, para garantizar resultados adecuados y un programa sin errores.

Un tipo de dato es una clasificación que describe los tipos de operaciones que se pueden realizar sobre él sin producir un error en la programación.

Las operaciones que se pueden usar para crear, transponer y aplicar la variable a otro cálculo están determinadas por el tipo de datos. El término "**fuertemente tipado**" se refiere a un lenguaje de programación que solo permite el uso de variables de acuerdo con sus tipos de datos.

2.1.1 Tipos de datos:

Cada lenguaje de programación es capaz de manejar una amplia variedad de datos. Sin embargo, siempre tendremos acceso a los tipos de datos básicos.

Estos están incluidos en el lenguaje de programación, lo que nos permite realizar tareas más difíciles. Veremos los tipos de datos más típicos que están presentes en la mayoría de los lenguajes de programación, como Java y Python.

Tipo de datos para números: int.

Dado que es importante resolver una serie de problemas, los números generalmente se representan en un lenguaje de programación de varias maneras:

- ¿Qué **tamaño** emplearemos?

Tech Hub <FP> euroformac

- Si los números estarán en **forma decimal**.
- Si el resultado va a ser **negativo**.

Comenzaremos con el tipo primitivo **int**. Cualquier número, positivo o negativo, sin decimales se representa con este tipo de datos.

Hay algunos lenguajes, como Pascal, que lo escriben como **integer**, aunque con frecuencia se escribe como int en el código fuente.

```
edad = 30
type(edad)   # Comando que nos indica el tipo de datos
<class 'int'> #resultado de la salida de comando Type.
```

Double o float son los tipos de datos de números reales.

Los tipos de datos **double** o **float** se utilizan normalmente cuando se trabaja con números decimales.

Este es un número de "coma flotante", como decimos. Según el nivel de precisión decimal que desees, tendrás que declararlo de un tipo u otro.

El tipo **double** tiene un rango de +/- 1.8 * 10^-308 a +/- 1.8 * 10^308.

El tipo **float** tiene un rango de +/- 3.4 * 10^-38 a +/- 3.4 * 10^38.

Otra forma de saber si un número es flotante o doble es verlo con la letra F o la letra D.

Por ejemplo 4.89F.

```
altura=1.85
type(altura) #Comando que nos indica el tipo de datos
<class 'float'>
```

Números complejos

Definición

En Python, los números complejos se pueden representar utilizando la función `complex()`. Un número complejo consta de una parte real y una parte imaginaria, donde la parte imaginaria se denota con la unidad imaginaria `j` o `J`.

Aquí tenemos un ejemplo de cómo se define un número complejo en Python:

```
numero_complejo = complex(3, 4)
print(numero_complejo)
```

En este caso, el número complejo se define con una parte real de 3 y una parte imaginaria de 4. Al imprimir `numero_complejo`, obtendremos el resultado:

(3+4j)

Podemos acceder a la parte real e imaginaria de un número complejo utilizando los atributos `real` e `imag`, respectivamente. Aquí tienes un ejemplo:

```
parte_real = numero_complejo.real
parte_imaginaria = numero_complejo.imag
print(parte_real)            # Imprime 3.0
print(parte_imaginaria)      # Imprime 4.0
```

Además, podemos realizar operaciones matemáticas con números complejos en Python, como suma, resta, multiplicación y división. Python proporciona funciones y operadores para trabajar con números complejos. Aquí tenemos algunos ejemplos:

```
# Suma de números complejos
resultado_suma = complex(2, 3) + complex(1, 2)
print(resultado_suma)       # Imprime (3+5j)

# Resta de números complejos
resultado_resta = complex(5, 7) - complex(2, 3)
print(resultado_resta)      # Imprime (3+4j)

# Multiplicación de números complejos
resultado_mult = complex(2, 1) * complex(3, 4)
print(resultado_mult)       # Imprime (-5+10j)

# División de números complejos
resultado_div = complex(4, 5) / complex(2, 1)
print(resultado_div)        # Imprime (2.6+0.8j)
```

Tipos de datos de cadena: char y string (str).

En Python, no existe un tipo de datos de "char" como en algunos otros lenguajes de programación. En su lugar, el tipo de datos de cadena en Python se representa con el tipo `str`, que se utiliza para almacenar secuencias de caracteres.

Aquí tenemos una breve definición de los tipos de datos de cadena en Python:

Definición

`str`: El tipo de datos `str` representa una cadena de caracteres. Puedes crear una cadena en Python utilizando comillas simples ('') o comillas dobles ("").

Por ejemplo:

```
cadena1 = 'Hola, mundo!'
cadena2 = "¡Hola, Python!"
```

Las cadenas en Python son inmutables, lo que significa que no podemos modificar los caracteres individuales de una cadena después de haberla creado. Sin embargo, puedes realizar diversas operaciones y manipulaciones de cadenas, como concatenación, extracción de subcadenas, búsqueda y reemplazo de caracteres, entre otros.

```
# Concatenación de cadenas
cadena_concatenada = cadena1 + ' ' + cadena2
print(cadena_concatenada)    # Imprime "Hola, mundo! ¡Hola,
Python!"

# Extracción de subcadenas
subcadena = cadena1[0:5]
print(subcadena)             # Imprime "Hola,"

# Búsqueda y reemplazo de caracteres
cadena_modificada = cadena1.replace('mundo', 'Python')
print(cadena_modificada)     # Imprime "Hola, Python!"
```

Además, Python proporciona una amplia variedad de métodos y funciones incorporadas para trabajar con cadenas, como `len()`, `lower()`, `upper()`, `split()`, entre otros, que nos permiten realizar diferentes operaciones y manipulaciones de cadenas.

Es importante tener en cuenta que las cadenas en Python son secuencias de caracteres Unicode, lo que significa que podemos trabajar con caracteres de diferentes idiomas y símbolos especiales sin problemas. Esto brinda una gran flexibilidad en el manejo de cadenas en Python.

Importante

Normalmente, escribimos el carácter Unicode conocido como "char" entre comillas simples ('').

Sin embargo, solemos identificar un carácter como una cadena si es una cadena de caracteres, que son caracteres que se siguen uno tras otro para formar una secuencia.

Para distinguirlo de char, el tipo de string (str) debe estar entre comillas dobles (“ “), aunque puede variar según el lenguaje de programación.

En resumen, en Python, utilizamos el tipo de datos `str` para representar cadenas de caracteres, y no hay un tipo de datos específico para un solo carácter como "char" en otros lenguajes. Las cadenas en Python son inmutables y proporcionan numerosas operaciones y funciones para manipular y trabajar con ellas.

Tipo de datos Array (Matriz o Vector).

Definición

El término "array" se utiliza ampliamente en la programación para referirse a una estructura de datos que almacena elementos del mismo tipo de manera contigua en la memoria. Los **arrays** permiten acceder a los elementos mediante un índice numérico y ofrecen eficiencia en términos de acceso y manipulación de datos.

A continuación, presentamos algunos tipos de datos "array" que se utilizan en diferentes lenguajes de programación:

- **Array unidimensional**: También conocido como vector, es una colección de elementos del mismo tipo dispuestos en una sola dimensión. Los elementos se acceden mediante un índice. Ejemplos de lenguajes que admiten arrays unidimensionales son C, C++, Java y Python.
- **Array bidimensional**: También conocido como matriz, es una estructura de datos con elementos organizados en filas y columnas. Se utiliza para representar tablas, matrices y estructuras similares. Los

elementos se acceden utilizando dos índices: uno para la fila y otro para la columna. Lenguajes como C, C++, Java y Python admiten arrays bidimensionales.

- **Array multidimensional**: Es una extensión del array bidimensional y permite almacenar elementos en más de dos dimensiones. Por ejemplo, un array tridimensional se utiliza para representar cubos o volúmenes. Lenguajes como C, C++, Java y Python admiten arrays multidimensionales.

Es importante tener en cuenta que la implementación y características de los arrays pueden variar según el lenguaje de programación. Algunos lenguajes proporcionan funcionalidades adicionales y métodos específicos para manipular arrays, como la capacidad de ordenar, buscar o realizar operaciones matemáticas en los elementos del array.

En resumen, los tipos de datos "array" se utilizan en la programación para almacenar elementos del mismo tipo de manera contigua en la memoria. Los arrays unidimensionales, bidimensionales y multidimensionales son algunos ejemplos comunes de arrays utilizados en diferentes lenguajes de programación.

A continuación, mostramos un ejemplo sencillo de un array en Python es el siguiente:

```
# ingredientes declarados como variables independientes
ingrediente1 = "leche"
ingrediente2 = "harina"
ingrediente3 = "azucar"
# Ingredientes añadidos en la lista
ingredientes = [ingrediente1, ingrediente2, ingrediente3]
```

En este caso, tendríamos una lista de ingredientes, a cada uno de los cuales le asignamos un valor fijo (ingrediente1, 2, 3, etc.). Sin embargo, si la lista es demasiado larga, se puede filtrar usando una matriz de Python para encontrar un elemento específico. Podemos agrupar tantos valores como quieras seleccionándolos por el número que tienen asignado.

Acceder a un elemento específico

Para acceder a un elemento concreto, utiliza el número de índice. Este es un ejemplo en el que primero se consulta el valor del primer elemento:

ingredientes[0]

'leche'

En el segundo paso, se introduce un nuevo valor en el array:

```
ingredientes[0]="crema"
ingredientes[0]
'crema'
```

La longitud de un array en Python

La longitud de un array en Python debe determinarse de antemano y no puede modificarse más adelante. Para establecer la longitud, seleccionaremos el valor más alto de los números de índice previstos y lo aumentamos en 1. Para obtener la longitud del array en Python, usaremos la función " **len ( )"**. Aquí tenemos un ejemplo:

```
numero_ingredientes = len(ingredientes)
numero_ingredientes
3
```

Añadir elementos a un array en Python

Para añadir elementos a nuestro array en Python, lo mejor es utilizar el método " append ( )" . Este funciona de la siguiente manera:

```
ingredientes.append("sal")
ingredients[3] #tenemos en cuenta que la posición del
primer elemento es la cero.
'sal'
ingredientes

['crema', 'harina', 'azucar', 'sal']
```

Eliminar elementos del array con pop ( ) o remove ( )

Existen dos formas de eliminar elementos de un array en Python. El primer método es **"pop ( )"**. Si quieres eliminar el tercer ingrediente (" sugar") del conjunto de ingredientes introducido anteriormente, utiliza el siguiente código:

```
tercero = ingredientes.pop(2)
tercero
'azucar'
ingredientes
['crema', 'harina', 'sal'] # valores que restan en el array
tras la eliminación del elemento "sugar".
```

Hay que tener en cuenta que se empieza a contar desde 0, por lo que el primer elemento toma el valor 0, el segundo el valor 1, y así sucesivamente.

El segundo método para eliminar el tercer ingrediente del array en Python es **"remove ( )"**. Este tiene el siguiente aspecto:

```
ingredientes.remove("crema")
ingredientes
```

```
['harina', 'sal']
```

De esta manera, el valor "crema" se extrae del array. Sin embargo, es importante saber que este método solo elimina el valor correspondiente la primera vez que aparece. Si el valor vuelve a aparecer de nuevo en el transcurso de la lista, no será eliminado de forma automática.

Crear un bucle de elementos en un array

En Python podemos crear un bucle de elementos del array con un bucle " for in". Con el siguiente comando se solicita asignar un ingrediente cada elemento del array " ingredientes" de Python:

```
for ingrediente in ingredientes:
    print (ingrediente)
#salida por pantalla
harina
sal
```

Lista de diferentes métodos para arrays en Python

Para un procesamiento más rápido y específico de estos arrays, deberíamos echar un vistazo a los diferentes métodos proporcionados por Python. Los siguientes métodos nos ayudarán a aprender más lenguajes de programación y trabajar con arreglos en Python:

Método	Descripción
append	Añade un elemento al final de la lista
clear	Este método elimina todos los elementos de la lista
copy	Produce una copia de toda la lista

count	Da como resultado el número exacto de elementos con un valor determinado
extend	Añade todos los elementos de una lista al final de un array en Python
index	Muestra el número de índice del primer elemento con un valor determinado
insert	Añade un elemento en una posición determinada
len	Se determina la longitud de un array en Python
pop	Se elimina un elemento en una posición determinada
remove	Elimina el primer elemento con un valor determinado
reverse	Invierte el orden de los elementos de tu array
sort	Ordenar tu lista

Datos del tipo de datos booleano

Definición

El tipo primitivo **boolean** representa valores booleanos. Muestra si una condición es verdadera o falsa. Por lo general, solo tienen identificados los valores **true** (verdadero) o **false** (falso). Puede ser lo mismo que los números 0 y 1 en algunos idiomas. Dos valores cualesquiera, o estados, pueden representarse mediante una variable.

Por ejemplo, podríamos discutir si algo es verdadero o falso, encendido o apagado, sí o no, activado o no, etc. Para comparar valores se utilizan los siguientes operadores: AND, OR, XOR, y NOT.

Operador Booleano	Resultado	Condición
x AND y	True	Si tanto x como y son verdaderas
x AND y	False	Si x o y es falso
x OR y	True	Si x o y, o ambos x e y son verdaderos
x OR y	False	Si tanto x como y son falsos
x XOR y	True	Si solo x o y es verdadero

x XOR y	False	Si x e y son verdaderos o falsos
x XOR y	True	Si x es falso
NOT x	False	Si x es verdadero

Operaciones lógicas tipo boolenano

Tipos de datos Date, Time and Timestamp

Estos tipos de datos se utilizan para trabajar con datos que contienen fechas y horas.

Tipo de Dato	Valor
DATE	Año, mes y día
TIME	Hora, minuto y segundo
TIMESTAMP	Año, mes, día, hora, minuto, segundo y microsegundo

Tipo de datos NoneType

Cuando ningún valor es válido, o para indicar que una variable aún no tiene un valor significativo, el valor **None** se utiliza para representar estas situaciones.

El valor **None** tiene un tipo único llamado **NoneType** que lo distingue de todos los demás valores.

2.2. Algoritmos

Definición

Un algoritmo es un conjunto de instrucciones predeterminadas, secuenciales y restringidas que utiliza una computadora para resolver un problema, realizar un cálculo o realizar una tarea. En otras palabras, un algoritmo es un conjunto de instrucciones que deben seguirse paso a paso para lograr un objetivo. Para resolver un problema, se deben tomar una serie de pasos secuenciales comenzando con el estado inicial y la información.

El primer paso en la programación es crear un algoritmo y luego comenzar a escribir el código. Para indicarle a la máquina qué hacer, primero debemos determinar cómo resolver el problema (definir el algoritmo en el ordenador). De esta manera, un programa de ordenador no sería más que una colección de algoritmos que han sido ordenados y codificados en un lenguaje de programación para que puedan usarse para ejecutar programas en los ordenadores.

Sin embargo, el estudio de las matemáticas, la lógica y la computación no implica exclusivamente el estudio de los algoritmos. En nuestra vida diaria, usamos una variedad de algoritmos para resolver problemas. Los manuales de instrucciones y los recetarios son algunos de los ejemplos más típicos.

Un algoritmo tiene estos tres componentes:

1. **Input (entrada).** Información que proporcionamos al algoritmo para que pueda trabajar para dar la solución deseada.
2. **Proceso.** Una serie de acciones que se toman para que la situación se resuelva utilizando los datos de entrada.
3. **Output(salida).** Resultados de los valores de entrada que se cambian durante el proceso.

De esta manera, un algoritmo informático comienza con un estado inicial y algunos valores de entrada, pasa por una serie de pasos posteriores y finalmente llega a un estado final en el que ha encontrado una solución.

Características de un algoritmo.

Los algoritmos enumeran una serie de rasgos compartidos. Son:

- Precisos. Objetivos y bien definidos.
- Organizados. Proporcionan un procedimiento claro y detallado para llegar a la respuesta.
- Finitos. Tienen un número particular de pasos.
- Concretos. Para la situación o problema planteado, presentan una solución detallada.
- Definidos. Cuando se le da la misma entrada, el mismo algoritmo debería producir la misma salida.

Representación de algoritmos

Las dos herramientas más utilizadas para describir algoritmos son:

2.2.1 Diagramas de flujo

Un diagrama de flujo es una representación visual de los pasos lógicos o algoritmos de un programa o proceso. Se utiliza como una herramienta de diseño y comunicación para mostrar cómo fluye la información y cómo se toman las decisiones en un programa.

Aquí hay una explicación de los elementos básicos de un diagrama de flujo:

- **Símbolos**: Los diagramas de flujo utilizan una serie de símbolos estándar para representar diferentes acciones y decisiones. Algunos de los símbolos comunes incluyen:
  - Inicio/Fin: Representado por un óvalo o un rectángulo redondeado. Indica el inicio o final del programa o proceso.
  - Proceso: Representado por un rectángulo. Representa una acción o proceso específico que se lleva a cabo en el programa.
  - Decisión: Representado por un rombo. Se utiliza para mostrar una bifurcación o una decisión en el programa. Por lo general, se utiliza una pregunta para indicar la decisión que se debe tomar.
  - Conector: Representado por un círculo pequeño con un número o letra dentro. Se utiliza para conectar diferentes partes de un diagrama de flujo cuando no hay suficiente espacio para colocar todos los símbolos en la página.
  - Flechas: Las flechas se utilizan para mostrar la dirección del flujo en el diagrama, indicando el orden secuencial de las acciones o decisiones.
- **Conexiones**: Las acciones y decisiones en un diagrama de flujo están conectadas por flechas que indican el flujo de ejecución del programa. Las flechas muestran la dirección en la que se ejecutan las acciones, desde el símbolo anterior hasta el siguiente.
- **Secuencia**: Los símbolos se organizan en secuencia para mostrar el orden en que se ejecutan las acciones. El flujo de ejecución normalmente sigue una secuencia de arriba hacia abajo y de izquierda a derecha.
- **Control de flujo:** Las decisiones en un programa se representan mediante símbolos de decisión (rombos). Estos símbolos tienen una pregunta o condición que se evalúa y se toma una dirección basada en el resultado. Por ejemplo, si una condición es verdadera, el flujo se dirige hacia un camino específico; de lo contrario, se sigue otro camino.
- **Bucles:** Los bucles o iteraciones se representan utilizando símbolos especiales en el diagrama de flujo. Pueden indicar que ciertas acciones se repiten hasta que se cumple una condición específica.

Tech Hub <FP> euroformac

- Un **diagrama de flujo** ayuda a comprender visualmente el flujo y la lógica de un programa o proceso. Puede ser una herramienta útil para el diseño, la depuración y la comunicación de algoritmos y lógica de programación.

	Inicio/Final Se utiliza para indicar el inicio y el final de un diagrama. De inicio solo puede salir una línea de flujo y al final solo debe llegar una línea.		**Decisión** Indica la comparación de dos datos y dependiendo del resultado lógico (falso o verdadero) se toma la decisión a seguir un camino u otro del diagrama
	Entrada / Salida Entrada/Salida de datos por cualquier dispositivo (scanner, lector de código de barras, microfono, etc)		**Impresora/Documento** Indica la presentación de uno o varios resultados en forma impresa.
	Entrada por teclado Indica que el PC debe esperar a que el usuario teclee un dato que se guardará en una variable o constante		**Pantalla** Instrucción de presentación de mensajes o resultados en pantalla.
	Acción / Proceso Indica una acción o instrucción general que debe realizarse (operaciones aritméticas, asignaciones, etc.)		**Conector Interno** Indica el enlace de dos partes de un diagram dentro de una misma página.
	Flujo/Flechas de dirección Indica el seguimiento lógico del diagrama.También indica el sentido de ejecución de las operaciones.		**Conector Externo** Indica el enlace de dos partes de un diagrama en páginas diferentes.

Símbolos en diagramas de flujo.

2.2.2 Pseudocódigos

Definición

Los pseudocódigos describen un algoritmo de forma similar a un lenguaje de programación, pero con mayor elasticidad, similar al lenguaje natural.

No es un lenguaje de programación real, sino más bien una representación de los pasos y la lógica del programa utilizando instrucciones simples y palabras clave.

El objetivo principal del pseudocódigo es ayudarnos a planificar y diseñar programas antes de comenzar a escribir código en un lenguaje de programación específico. Al utilizar el pseudocódigo, podemos centrarnos en la lógica y la estructura del programa sin preocuparnos por la sintaxis específica del lenguaje de programación.

El pseudocódigo se basa en un conjunto de convenciones y reglas simples. Aunque no hay un estándar estricto, generalmente sigue estas pautas:

- **Expresividad:** El pseudocódigo se escribe en un lenguaje cercano al lenguaje natural, utilizando palabras clave y frases simples para describir las acciones y los pasos del algoritmo. Esto facilita la comprensión tanto para los programadores como para las personas no técnicas que puedan leer el pseudocódigo.
- **Instrucciones y estructuras de control:** El pseudocódigo utiliza estructuras de control como bucles (por ejemplo, "para", "mientras") y condicionales (por ejemplo, "si", "sino") para controlar el flujo del programa y tomar decisiones basadas en condiciones específicas.
- **Asignaciones de variables:** Se utilizan instrucciones para asignar valores a variables y realizar operaciones matemáticas básicas. Por ejemplo, se pueden usar declaraciones como "asignar x = 5" o "x = x + 1" para asignar valores a una variable llamada "x" o actualizar su valor.
- **Comentarios:** Se pueden incluir comentarios en el pseudocódigo para explicar partes del algoritmo o proporcionar aclaraciones adicionales. Los comentarios generalmente se escriben utilizando un símbolo especial (como "//" o "#") al comienzo de la línea.

Es importante tener en cuenta que el pseudocódigo no se ejecuta en un entorno de programación real. Su objetivo principal es describir y comunicar la lógica y la estructura del programa. Una vez que el algoritmo está bien diseñado en pseudocódigo, se puede traducir a un lenguaje de programación real para su implementación.

El pseudocódigo es una herramienta útil para planificar y diseñar programas antes de comenzar a escribir código. Permite a los programadores tener una visión clara de la lógica del programa y facilita la comunicación

Tech Hub <FP> euroformac

Sabías que

Algunas características del pseudocódigo incluyen:

Legibilidad: El pseudocódigo se escribe de manera clara y legible, utilizando un lenguaje sencillo y comprensible para cualquier persona familiarizada con la programación.

Abstracción: El pseudocódigo se enfoca en la lógica y los pasos generales del algoritmo, sin entrar en detalles de implementación o sintaxis específica de un lenguaje de programación en particular.

Flexibilidad: No hay reglas estrictas en cuanto a la sintaxis o la estructura del pseudocódigo. Se permite utilizar un estilo libre para expresar la lógica del algoritmo.

Comentarios: Es común incluir comentarios en el pseudocódigo para explicar el propósito de ciertas partes del algoritmo o proporcionar aclaraciones adicionales.

Ver enlace.

Ejemplo de Pseudocódigo: Calcular una altura en pulgadas (1 pulgada=2.54 cm) y pies (1 pie=12 pulgadas), a partir de la altura en centímetros, que se introduce por el teclado.

```
Inicio
     IMPRIMIR 'Introduce la altura en centímetros:'
     LEER: altura
     CALCULAR pulgadas=altura/2.54
     CALCULAR pies=pulgadas/12
     IMPRIMIR 'La altura en pulgadas es:', pulgadas
     IMPRIMIR 'La altura en pies es:', pies
Fin
```

2.3. Ejemplos de diferentes tipos de algoritmos

Los algoritmos se pueden categorizar de diferentes maneras según varios factores. Si lo hacen mediante secuencias lógicas o cálculos matemáticos, también se distingue entre algoritmos cuantitativos y cualitativos en función del resultado que proporcionan y también se clasifican en computacionales y no computacionales dependiendo de si se necesita o no una computadora para resolverlos.

Pero si examinamos su propósito (lo que hace) y el enfoque del problema (cómo lo hace), descubrimos una amplia variedad de otros algoritmos. Se destacan las siguientes cinco categorías de algoritmos informáticos.

2.4. Algoritmos de búsqueda

Los algoritmos de búsqueda localizan uno o más elementos en una estructura de datos que tienen un conjunto particular de características.

Existen numerosas categorías de búsqueda, pero se destacan las siguientes:

- **Búsqueda lineal o secuencial**. Durante el cual se compara el elemento buscado con cada miembro del conjunto hasta encontrarlo o hasta comparar todos los elementos.

```
""" Búsqueda lineal.
 Si x está en lista devuelve su posición en lista, de lo
 contrario devuelve -1.
"""


# Función del Algoritmo: se recorren uno a uno los
elementos de la lista
# y se los compara con el valor x buscado.
def busqueda_lineal(lista, x):
    i = 0  # i tiene la posición actual en la lista,
comienza en 0

    # El ciclo recorre todos los elementos de la lista


for z in lista:

    # Si z es igual a x, devuelve el valor de i
    if z == x:
        return i
    i = i + 1
    # si salió del ciclo sin haber encontrado el valor,
devuelve -1
return -1

Resultado:
>> > busqueda_lineal([1, 4, 54, 3, 0, 34, 23, 12], 2)
-1
>> > busqueda_lineal([1, 4, 54, 3, 0, 34, 23, 12], 23)
6
>> > busqueda_lineal([1, 4, 54, 3, 0, 34, 23, 12], 4)
1
```

- **Búsqueda binaria.** En un conjunto ordenado, cuando comparamos dos elementos, los comparamos con el elemento del medio; si no son iguales, la búsqueda pasa al medio, donde podría estar, y así

sucesivamente, en espacios entre elementos progresivamente más pequeños.

```
#Este es el vector que el algoritmo usara para buscar
cualquier dato
vector = [3, 5, 8, 9, 10, 22, 45, 500, 900, 4253]

#La variable puntero será el inicio del vector, que es 0
puntero = 0

#vectorLen contiene la longitud del vector
vectorLen = len(vector)

#La varieable encontrado cambiara su valor, y asi el
algoritmo sabre que hacer
encontrado = False

#Le pedimos al usuario una entrada de un entero
numero = int(input("Ingresar el dato que desea buscar:
"))

#Creamos un bucle que no se detenga hasta que encontrado
sea diferente de False
#Y que puntero sea menor o igual que vectorLen
while not(encontrado) and puntero <= vectorLen:

 #Creamos la variable mitad
 mitad = int((puntero+vectorLen) / 2)

 #Si numero es igual que el indice mitad en vector
 if numero == vector[mitad]:
  #Encontado sera igual a True
  encontrado = True

 elif numero < vector[mitad]:
  #vectorLen sera igual que mitad - 1
  vectorLen = mitad - 1
 #De lo contrario
 else:
  #Puntero sera igual que mitad + 1
  puntero = mitad + 1
```

```
#Si encontrado es True
if(encontrado):

#Mostramos un mensaje con la posicion del Dato en el
vector
 print("El dato se encuentra en la posición ",
str(mitad+1))

 #Mostramos el vector ordenado
 print(vector)

#De lo contrario
else:
        #Mostramos un mensaje
 print("El dato no se encontró")

Resultado:
Ingresar el dato que desea buscar: 8
El dato se encuentra en la posición 3
[3, 5, 8, 9, 10, 22, 45, 500, 900, 4253]
```

2.5 Algoritmos para ordenar

Reorganizan los componentes de la lista de acuerdo con la relación de orden. Predomina el orden **léxico** y el orden **numérico**. La ordenación efectiva optimiza el uso de algoritmos, como la búsqueda, y también facilita la producción de resultados que son comprensibles tanto para humanos como para máquinas.

Los siguientes son algunos algoritmos de clasificación:

- **Ordenamiento de burbuja**. compara cada elemento de la lista que se ordenará con el elemento que le sigue y, si es necesario, invierte el orden de los elementos. La lista completa se examina varias veces después de que se decide que no se requieren más intercambios. El siguiente es su código de Python:

```
from random import sample

# Importamos un Método de la biblioteca random para generar
listas aleatorias

lista = list(range(100))  # Creamos la lista base con
números del 1 al 100
```

```
# Creamos una lista aleatoria con sample
# (8 elementos aleatorios de la lista base)
vectorbs = sample(lista, 8)


def bubblesort(vectorbs):
    """Esta función ordenara el vector que le pases como
argumento con el Método de Bubble Sort"""

    # Imprimimos la lista obtenida al principio
(Desordenada)
    print("El vector a ordenar es:", vectorbs)
    n = 0  # Establecemos un contador del largo del vector

    for _ in vectorbs:
        n += 1  # Contamos la cantidad de caracteres dentro
del vector

    for i in range(n - 1):
        # Le damos un rango n para que complete el proceso.
        for j in range(0, n - i - 1):
            # Revisa la matriz de 0 hasta n-i-1
            if vectorbs[j] > vectorbs[j + 1]:
                vectorbs[j], vectorbs[j + 1] = vectorbs[j +
1], vectorbs[j]
            # Se intercambian si el elemento encontrado es
mayor
            # Luego pasa al siguiente
    print("El vector ordenado es: ", vectorbs)


bubblesort(vectorbs)
## El vector a ordenar es: [53, 1, 38, 93, 74, 83, 4, 18]
## El vector ordenado es:  [1, 4, 18, 38, 53, 74, 83, 93]
```

- **Ordenamiento rápido o quickshort**. Según sean mayores o menores que el elemento que estemos usando como referencia, elegimos un elemento del conjunto (el pivote) y movemos el resto de los elementos a cada uno de sus lados. Cada subconjunto está sujeto al mismo proceso. Su código en Python sería el siguiente:

```
from random import sample

# Importamos un Método de la biblioteca random para generar
listas aleatorias

lista = list(range(100))  # Creamos la lista base con
```

```
números del 1 al 100

# Creamos una lista aleatoria con sample
# (8 elementos aleatorios de la lista base)
vectorquick = sample(lista, 8)


def quicksort(vectorquick, start=0, end=len(vectorquick) -
1):
    """Esta función ordenara el vector que le pases como
argumento
    con el Método Quick Sort"""

    # Imprimimos la lista obtenida al principio
(Desordenada)
    print("El vector a ordenar con quick es:", vectorquick)

    def quick(vectorquick, start=0, end=len(vectorquick) -
1):

        if start >= end:
            return

        def particion(vectorquick, start=0,
end=len(vectorquick) - 1):
            pivot = vectorquick[start]
            menor = start + 1
            mayor = end

            while True:
                # Si el valor actual es mayor que el pivot
                # está en el lugar correcto (lado derecho
del pivot) y podemos
                # movernos hacia la izquierda, al siguiente
elemento.
                # También debemos asegurarnos de no haber
superado el puntero bajo, ya que indica
                # que ya hemos movido todos los elementos a
su lado correcto del pivot
                while menor <= mayor and vectorquick[mayor]
>= pivot:
                    mayor = mayor - 1

                # Proceso opuesto al anterior
                while menor <= mayor and vectorquick[menor]
<= pivot:
                    menor = menor + 1

                # Encontramos un valor sea mayor o menor y
```

```
que este fuera del arreglo
                # ó menor es más grande que mayor, en cuyo
caso salimos del ciclo
                if menor <= mayor:
                    vectorquick[menor], vectorquick[mayor]
= vectorquick[mayor], vectorquick[menor]
                    # Continua el bucle
                else:
                    # Salimos del bucle
                    break

            vectorquick[start], vectorquick[mayor] =
vectorquick[mayor], vectorquick[start]

            return mayor

        p = particion(vectorquick, start, end)
        quick(vectorquick, start, p - 1)
        quick(vectorquick, p + 1, end)

    quick(vectorquick)
    print("El vector ordenado con quick es:", vectorquick)


quicksort(vectorquick)
## El vector a ordenar con quick es: [83, 30, 94, 0, 58,
40, 74, 16]
## El vector ordenado con quick es: [0, 16, 30, 40, 58, 74,
83, 94]
```

2.6. Algoritmos voraces

Definición

Un algoritmo voraz (también conocido como algoritmo greedy, en inglés) es un enfoque de diseño de algoritmos que resuelve un problema tomando decisiones locales óptimas en cada etapa, con la esperanza de alcanzar una solución global óptima. En otras palabras, en cada paso del algoritmo, se elige la opción más favorable en ese momento, sin considerar las posibles consecuencias futuras.

La principal característica de un algoritmo voraz es que toma decisiones basadas únicamente en la información disponible en ese momento y sin

revisar o modificar las decisiones tomadas previamente. Esto puede llevar a una solución subóptima o incluso incorrecta en algunos casos, ya que no se realiza un análisis exhaustivo de todas las posibilidades.

Ejemplos de algoritmos voraces:

Debido a que sólo consideran la información de las decisiones pasadas y no las posibles futuras, estos algoritmos ocasionalmente no logran encontrar la mejor solución general. Las siguientes son algunas situaciones en las que los algoritmos voraces producen los mejores resultados.

- **Problema de mochila fraccional (KP)**. Debemos elegir qué elementos poner en la mochila para que pueda llevar el mayor peso posible sin pasarse de su límite de peso. Tenemos una colección de artículos, cada uno de los cuales tiene un valor y un peso correspondiente.
- **El algoritmo de Dijkstra**. Se utiliza para encontrar la ruta más corta a lo largo de un gráfico con bordes ponderados desde un vértice de origen hasta todos los demás vértices.
- **La Codificación de Huffman.** Técnica de compresión de datos sin pérdida de información que asigna a los caracteres de un mensaje un código de longitud variable en función de su frecuencia de aparición. Los códigos más cortos corresponderán a frecuencias más altas.

2.7. Programación dinámica

En el enfoque de programación dinámica para la resolución de problemas, descomponemos un problema complejo en problemas más pequeños, calculamos sus soluciones y luego las almacenamos para que no tengamos que volver a calcularlas más tarde para encontrar la solución general. Al hacer que la recursividad sea más eficiente, la programación dinámica acelera la ejecución del algoritmo.

Naturalmente, un problema necesita subestructuras óptimas y subproblemas superpuestos para poder aplicarlo. Es decir, el problema se puede dividir en subproblemas que luego se utilizan para proporcionar la solución general, y es posible combinar las mejores soluciones a los subproblemas para encontrar la mejor solución al problema general.

Esta es una situación donde se aplica:

La sucesión de Fibonacci. La secuencia de números comienza con "0" y "1", y cada número que sigue es el resultado de sumar los dos anteriores. Se define por una relación de recurrencia.

2.8. Algoritmos probabilísticos

Es un método cuya lógica incorpora una fuente de aleatoriedad. Al seleccionar aleatoriamente algunas de las entradas, encuentra una solución que puede no ser perfecta, pero que es suficiente para el problema en cuestión.

Debido a que se pueden obtener múltiples soluciones a partir de los mismos datos, algunas de las cuales son incorrectas, se utiliza cuando hay escasez de tiempo o memoria y cuando se puede aceptar una buena solución promedio. El algoritmo se ejecuta repetidamente utilizando varias submuestras aleatorias para aumentar la probabilidad de que produzca la respuesta correcta y se comparan los resultados.

En general, los algoritmos probabilísticos se dividen en dos categorías:

- **El algoritmo de Montecarlo**. Existe una pequeña posibilidad de que no encuentre una solución dependiendo de la entrada. Al extender el tiempo de cálculo, se puede disminuir la probabilidad de error.
- **Fórmula de Las Vegas**. Se lleva a cabo en un tiempo determinado. Será preciso si encuentra una solución dentro de ese tiempo, pero también es posible que no la encuentre si se acaba el tiempo.

2.9 Ejemplo sencillo de creación de un algoritmo:

Ejemplo de un algoritmo sencillo en Python para calcular la circunferencia de un número:

```
import math #  se carga esta librería donde se detalla
el valor de la constante pi
# El valor puede venir de cualquier lugar
diametro = float(input("Ingresa el diámetro: "))
circunferencia = diametro * math.pi
radio = diametro / 2
print(f"La circunferencia es {circunferencia} y el
radio es {radio}")
```

Ejecución del algoritmo:

```
C:\Users\PycharmProjects\pythonProject\venv\Scripts\py
thon.exe
C:\Users\PycharmProjects\pythonProject\Testing.py

Ingresa el diámetro: 10

La circunferencia es 31.41592653589793 y el radio es
5.0

Process finished with exit code 0
```

2.10. Programas

Definición

Un programa es un conjunto de instrucciones escritas en un lenguaje de programación concreta que la computadora puede entender y usar para llevar a cabo una tarea o encontrar una solución. No podremos escribir el programa si no entendemos cómo resolver este problema. Como resultado, un buen programador siempre comienza haciendo un análisis amplio conocimiento del problema que espera resolver.

Lo primero que debemos tener en cuenta es que el objetivo de un programa es resolver un problema y que existen numerosos enfoques para un mismo problema.

Un programador puede crear varios programas para resolver el mismo problema cuando se le presenta el problema de una manera única.

Un programa cuenta con los siguientes componentes:

A continuación, se explica cada uno de estos elementos:

- **Entrada:** La entrada de un programa es la información o los datos que se proporcionan al programa antes de que comience su ejecución. Puede ser ingresada por el usuario a través del teclado, desde un archivo, desde una base de datos u otras fuentes. La entrada puede incluir números, texto, fechas, imágenes u otros tipos de datos que el programa necesita para realizar su tarea.
- **Proceso:** El proceso se refiere a las instrucciones o algoritmos que se ejecutan en el programa para manipular, procesar o transformar los datos de entrada. Estas instrucciones pueden incluir operaciones matemáticas, comparaciones lógicas, estructuras de control como bucles y condicionales, llamadas a funciones, entre otros. Durante el proceso, el programa utiliza los datos de entrada y realiza las operaciones necesarias para producir un resultado.
- **Salida:** La salida de un programa es el resultado o los datos generados después de que el proceso ha sido ejecutado. Puede ser mostrada al usuario a través de la pantalla, guardada en un archivo, enviada a una impresora o utilizada de alguna otra manera según los requisitos del programa. La salida puede ser cualquier tipo de información que el programa deba proporcionar como resultado final, como un cálculo, un informe, una visualización gráfica, etc. Ejemplo de la estructura de un programa en Python:

Vamos a crear un programa que calcule el total de una factura tras aplicarle el IVA. Esta función tomará el importe sin IVA y el porcentaje de IVA a aplicar y devolverá el valor total de la factura. Si se llama a la función sin que se le pase un porcentaje de IVA, se descontará el 21%.

#algoritmo o instrucciones de procesamiento de los datos de entrada

```
def invoice(amount, vat=21):
    """Función de aplica el IVA a una factura.
    Parametros
    amount: Es la cantidad sin IVA
    vat: Es el porcentaje de IVA
    Devuelve el total de la factura una vez aplicado el IVA
.
    """
    return amount + amount*vat/100
```

#Entrada de datos del programa:

```
    print(invoice(1000,10))#ejecución del programa con IVA
    diferente.
    print(invoice(1000))#ejecución con IVA 21%
```

```
#Salida de los datos por pantalla:
1100.0
1210.0
```

Tech Hub <FP> euroformac

3. Paradigmas de programación

Las ideas fundamentales detrás de los paradigmas de programación, o las diversas formas de generar código e instruir máquinas, computadoras y otros dispositivos, son fundamentales para el desarrollo de software.

Los programas informáticos ofrecen a los usuarios opciones para llevar a cabo diversas tareas utilizando hardware específico al mismo tiempo que intentan resolver problemas. Dependiendo del tipo de problema, se deben tomar diferentes enfoques para resolverlo. Un paradigma de programación puede ser más efectivo que otro.

Los paradigmas de programación se pueden considerar como un camino, ruta o guía que acelera el desarrollo mientras se adhiere a una estructura específica y un conjunto de reglas.

Cada paradigma tiene una estructura única, lo que da como resultado una variedad de paradigmas de programación.

Tipos de paradigmas de programación

Ahora examinaremos los dos paradigmas de programación diferentes y sus características:

Los lenguajes de programación **imperativos** son aquellos en los que el programador debe describir con precisión a la computadora la operación que desea realizar.

Los **declarativos** son lenguajes de programación en los que el programador le dice a la computadora qué hacer y qué resultado quiere, pero no cómo lograr esos objetivos. En este caso, es seguro decir que la máquina juega el papel principal en este paradigma.

La programación **imperativa**, donde ejecutamos algoritmos con acciones secuenciales, es actualmente el paradigma o enfoque de programación **más popular**. Este método de programación le da a la computadora todos los pasos que debe seguir para resolver un problema. La programación estructurada u orientada a objetos es otro ejemplo de estilos de programación que se pueden usar con la programación imperativa. El paradigma funcional o lógico contrasta fuertemente con la programación orientada a objetos.

Tech Hub <FP> euroformac

3.1. Programación imperativa

Este es el paradigma convencional de la programación, en el que se indica en el código cada instrucción paso a paso que debe realizar un programa. Pascal, C y C++ son algunos paradigmas de este tipo.

Numerosas estrategias que están subordinadas a este paradigma se pueden encontrar en la programación imperativa, estos son algunos ejemplos:

- **Programación Estructurada**

Estructuras condicionales, subrutinas y bucles se utilizan para controlar el flujo en este tipo de programación imperativa. Es mejor evitar el uso de saltos absolutos entre instrucciones.

El paradigma estructurado típico para la enseñanza de la programación es aquel en el que las instrucciones se leen de arriba a abajo y donde no es necesario atascarse en ideas o cosas complicadas (sólo concentrarse en dar órdenes consecutivas). C, Fortran y Pascal son ejemplos de este tipo de lenguaje.

- **Programación Procedimental.**

En este paradigma el código se divide en unidades más pequeñas y fáciles de administrar llamadas procedimientos o funciones en la programación de procedimientos. Como resultado de las llamadas a las funciones y procedimientos, el código se vuelve más claro y no necesita repetirse. Python y C son un par de ejemplos de este tipo de lenguaje.

- **Programación orientada a objetos.**

Cada componente de un programa es tratado como un objeto con sus propios atributos bajo el paradigma de programación orientada a objetos. El software puede ser moldeado por la interacción y el comportamiento de esos objetos.

Uno de los tipos más simples de programación, agregar un nuevo objeto, cambiar los atributos de un objeto existente o agregar funcionalidad a los objetos apropiados es todo lo que se requiere para implementar una nueva función. Lenguajes como Java o Ruby son ejemplos de este tipo.

3.2. Programación declarativa

A medida que el hardware mejora con el tiempo, también lo hace el software, lo que lleva al advenimiento de un paradigma de programación diferente llamado declarativo. En este tipo de programación se describe el resultado deseado y la búsqueda de la meta tiene prioridad sobre los medios para

lograrlo. La programación declarativa tiene un alto nivel de abstracción, lo que complica la comprensión del código.

La programación declarativa se puede abordar de tres maneras principales:

- **Programación Funcional**

Un programa funcional se compone de una serie de funciones, cada una de las cuales recibe datos, los procesa y luego devuelve un valor de salida. Este estilo de programación es perfecto para desarrollar algoritmos para aplicaciones técnicas y matemáticas (inteligencia artificial) y otros problemas complejos. Scala o LISP son lenguajes ilustrativos de este tipo.

- **Programación Lógica**

La programación lógica o predictiva se basa en la lógica matemática y sigue un conjunto de preceptos que se basan en suposiciones y hechos. Este tipo de programa de computadora utiliza los principios de la lógica matemática para abordar los problemas que encuentra. Haskell, Prolog y JavaScript son algunos ejemplos de este tipo de lenguaje.

- **Programación reactiva**

La programación reactiva nos permite ver cambios en un flujo de datos, como los cientos de comentarios que recibe un chat en vivo cada segundo o la información de ubicación en tiempo real que nos envía Google Maps.

Por lo tanto, la programación reactiva monitoriza estos flujos de datos y, cuando cambian, toma medidas.

Quedará muy claro con la ayuda de este ejemplo. Netflix no detiene la transmisión cuando la velocidad de Internet disminuye mientras transmite contenido desde el servicio; en cambio, la programación reactiva hace que el video se transmita con una calidad más baja.

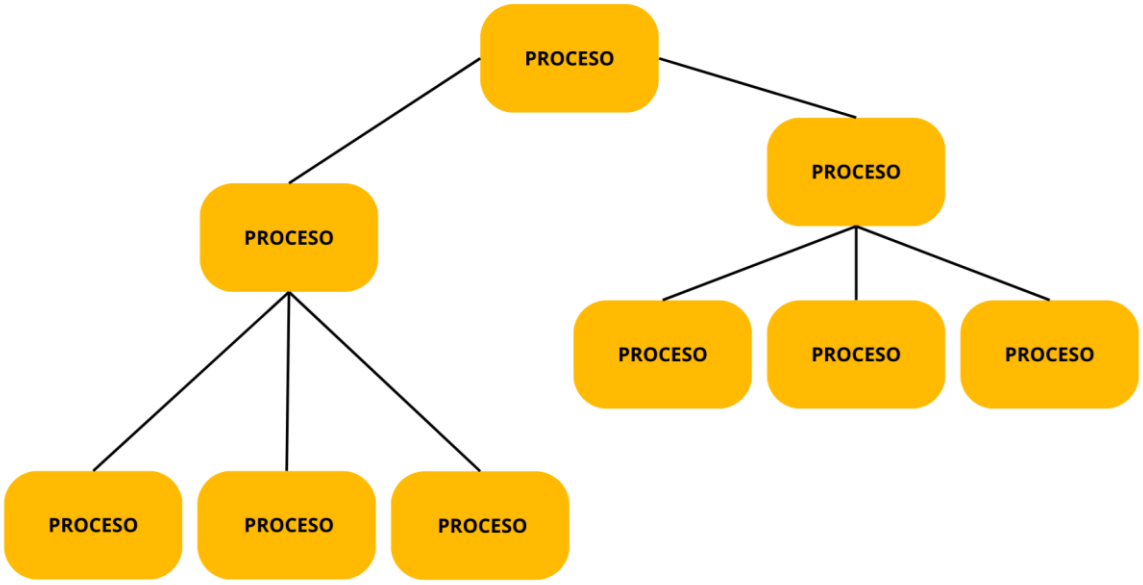
Se han discutido los paradigmas de programación, junto con los diversos tipos que existen. Los desarrolladores y programadores utilizan con frecuencia múltiples lenguajes de programación y diferentes paradigmas en la actualidad.

Dado que los diversos paradigmas son útiles para resolver varios problemas, es imposible decir cuál es superior a los demás porque esto depende del proyecto de programación y el problema en cuestión.

El aprendizaje de diversos lenguajes de programación que se basen en diversos paradigmas o formas de abordar y resolver problemas será necesario

a la hora de aprender a programar e iniciar una carrera profesional como desarrollador.

Mapa mental de resumen de ideas:



Esquema de paradigmas de programación

Comparativas de los paradigmas:

Paradigma de programación imperativa	Paradigma de programación declarativa
"¿Cómo?"	"¿Qué?"
Clásico	Moderno
Según este paradigma, un programa se compone de una serie de instrucciones que le indican a la computadora qué cálculos y acciones debe realizar y en qué orden	De acuerdo con este paradigma, un programa consta de instrucciones que especifican cómo debe responder un programa a una solicitud. El procedimiento de recursión gobierna los cálculos, que se realizan manipulando valores

Procede del latín “imperare”, que significa “ordenar”	Procede del latín “declarare”, que significa “denominar”
Se predetermina la vía de solución deseada	Se predetermina el resultado deseado

Paradigmas

Tech Hub <FP> euroformac

4. Lenguajes de programación

Los sistemas operativos se crean para facilitar que las personas interactúen con las computadoras.

Computadoras, como el **ENIAC** (Electronic Numerical Integrator And Calculator) se terminaron de construir a finales del año 1945, y llevaban una programación manual que se efectuó conectando cables y presionando interruptores.

Se utilizaron tarjetas perforadas para suministrar los datos. La programación para los cálculos típicos duraba entre 30 minutos a un día entero. Unos años más tarde nacen las instrucciones binarias (secuencias de unos y ceros) que son la base sobre la que se lleva a cabo la programación. Estas secuencias de ceros y unos representan una tecla activa o inactiva.

Este "Lenguaje de máquina" fue el término utilizado para describir este método de programación. Los primeros lenguajes simbólicos se crearon a principios de la década de 1950 como resultado del requisito de recordar secuencias para la planificación de las acciones típicas. Estas acciones tenían nombres cortos y simples que eran fáciles de recordar y asociar: ADD, SUB, MUL, CALL, etc. A este conjunto de instrucciones se le llamó **"lenguaje ensamblador".**

Los primeros lenguajes de alto nivel se crearon a fines de la década de 1950 y principios de la de 1960. Se denomina " código fuente" al código desarrollado por el programador mientras que se denomina "**código binario**" al código traducido a lenguaje máquina (unos y ceros).

Definición

Un lenguaje de programación está compuesto por símbolos y reglas que definen su estructura sintáctica y semántica.

El comportamiento físico y lógico de una máquina está controlado por el significado de las expresiones del lenguaje de programación empleado.

Aunque con frecuencia se usan indistintamente, "lenguaje de programación" y "lenguaje informático" no son lo mismo, se debe tener en cuenta que los lenguajes informáticos también incluyen lenguajes de programación.

Usando un lenguaje de programación, se puede definir con precisión los tipos de datos con los que debe trabajar una computadora, qué pasos debe tomar

sobre esos datos en una variedad de circunstancias y cómo se deben transmitir o almacenar estos datos.

CLASIFICACIÓN

Hay muchas clasificaciones diferentes para los lenguajes de programación, pero las principales son:

- Dependiendo de cuán abstracto sea un concepto.
- Según el modo de ejecución.
- De acuerdo con los respectivos paradigmas de programación que cada uno de ellos utiliza.

Según su nivel de abstracción:	Lenguajes de bajo nivel
	Lenguajes de nivel medio
	Leguajes de alto nivel
Según la forma de ejecución:	Lenguajes compilados
	Lenguajes interpretados
Según el paradigma de programación:	Lenguajes imperativos
	Lenguajes funcionales
	Lenguajes lógicos
	Lenguajes estructurados
	Lenguajes orientados a objetos

Clasificaciones para los lenguajes de programación

4.1. Máquinas y lenguajes de bajo nivel

El lenguaje de máquina está escrito en código (código de máquina) que la máquina (computadora) puede entender directamente, y sus instrucciones son cadenas binarias (0 y 1). "Lenguaje de máquina" se refiere al lenguaje específico de la computadora, mientras que "código de máquina" se refiere a cómo se escriben los diferentes lenguajes de máquina. Un lenguaje de bajo nivel es un lenguaje de programación que se aproxima a cómo funcionan las computadoras. Los lenguajes de nivel inferior son lenguajes de máquina. Este

nivel sigue al lenguaje ensamblador, porque la programación en lenguaje ensamblador trabaja directamente con los registros de memoria de la computadora. Programar en lenguajes de bajo nivel tiene la ventaja de ser más adaptable a nuestro hardware, además de poder alcanzar la máxima velocidad con el mínimo uso de memoria. Sin embargo, tiene desventajas significativas, como la incapacidad de escribir código independiente del dispositivo y los programas son más difíciles de programar y entender.

4.2. Lenguaje de nivel intermedio

En algunos programas de forma minoritaria se distinguen lenguajes de medio nivel como el lenguaje C, que presentan características que los hacen similares a los lenguajes de bajo nivel,
como gestión de punteros de memoria y registros, aunque presentan sintaxis, vocabulario y gramática de alto nivel.

4.3. Lenguajes de alto nivel y de muy alto nivel

Los lenguajes de programación de alto nivel tienen la ventaja de expresar algoritmos de manera consistente con las capacidades cognitivas humanas, en lugar de estar orientados a su implementación en máquinas. Los lenguajes de alto y bajo nivel requieren programación y conocimientos específicos del lenguaje (vocabulario, gramática, sintaxis) para implementar secuencias de declaraciones booleanas. Los lenguajes de alto nivel se crearon para que los usuarios comunes pudieran resolver algunos problemas informáticos simples de manera más fácil y rápida.

CON RESPECTO A LA FORMA DE EJECUCIÓN

Los procesadores de computadoras pueden comprender y ejecutar instrucciones de programas escritos en lenguaje máquina, que es un lenguaje predefinido para cada arquitectura.

Cualquier programa producido en un lenguaje de alto nivel se puede llevar a cabo de una de dos maneras:

- **Lenguajes compilados**: Se debe llamar a un traductor antes de poder usar el programa.
  El "compilador" se encarga de convertir el código fuente del programa inicial ("compilador") en el programa final equivalente expresado en lenguaje de máquina o ensamblador ("binario"). Los únicos

componentes necesarios para que un programa se ejecute son sus binarios ejecutables.

- **Lenguajes interpretados:** Cuando se usa un programa, se debe usar un "compilador", quien es responsable de traducir ("compilar") las instrucciones originales del programa ("código fuente") en código de máquina a medida que se usa.
  El código fuente original y el intérprete, siempre son necesarios para que el programa se ejecute. En la mayoría de los casos, los programas comerciales se distribuyen como código objeto compilado y no se incluye el código fuente. Esto se debe a que las empresas que desarrollan software comercial suelen considerar el código fuente como información confidencial y protegida.
  Al recibir un programa comercial, como un sistema operativo o una aplicación como Microsoft Office, no tendremos acceso al código fuente, lo que significa que no podremos modificarlo ni ver cómo está implementado internamente. Solo podemos ejecutar el programa según las funcionalidades y opciones proporcionadas por el software compilado.

Sabías que

Sin embargo, hay casos en los que las empresas eligen liberar el código fuente de sus programas comerciales. Esto se conoce como "software de código abierto". En el software de código abierto, el código fuente está disponible para que cualquier persona lo examine, modifique y distribuya según los términos de la licencia de código abierto correspondiente.

Es importante destacar que existen diferentes modelos de licencias de código abierto, como la Licencia Pública General de GNU (GPL), la Licencia MIT, la Licencia Apache, entre otras. Cada una de estas licencias establece diferentes condiciones y restricciones sobre cómo se puede utilizar, modificar y distribuir el código fuente.

En resumen, en el caso de los programas comerciales típicos, el código fuente no se proporciona, pero en el caso del software de código abierto, el código fuente está disponible para su modificación y redistribución bajo los términos de una licencia de código abierto específica.

Los lenguajes de programación a través del tiempo.

La historia de los lenguajes de programación es larga y diversa, con numerosos lenguajes que han surgido y evolucionado a lo largo de los años. Aquí exponemos un resumen de los hitos más importantes en esta historia.

- **Lenguaje máquina (década de 1940):** Los primeros ordenadores programables se programaban directamente en lenguaje máquina, que consiste en instrucciones escritas en código binario (1 y 0) entendidas por la máquina. Este lenguaje era muy difícil de aprender y propenso a errores.
- **Ensamblador (década de 1950):** Los lenguajes ensambladores fueron desarrollados para simplificar la programación en lenguaje máquina. Estos lenguajes utilizaban mnemónicos para representar instrucciones de bajo nivel, lo que facilitaba la programación y hacía que el código fuera más legible.
- **Fortran (1957):** El lenguaje Fortran (Formula Translation) fue el primer lenguaje de programación de alto nivel ampliamente utilizado. Fue desarrollado por IBM y se utilizó principalmente para aplicaciones científicas y de ingeniería. Fortran introdujo conceptos como las variables y las estructuras de control de flujo, y se mantuvo popular durante muchos años.
- **COBOL (1959):** COBOL (COmmon Business-Oriented Language) fue desarrollado para aplicaciones comerciales y empresariales. Su objetivo era facilitar la programación para usuarios no técnicos. COBOL se diseñó para ser legible por humanos y aún se utiliza en sistemas heredados en la actualidad.
- **Lisp (1958):** Lisp (List Processing) fue uno de los primeros lenguajes de programación de alto nivel y se enfocó en el procesamiento de listas y la manipulación simbólica. Lisp tuvo una gran influencia en el desarrollo posterior de lenguajes y se utiliza en aplicaciones como la inteligencia artificial.
- **ALGOL (1958):** ALGOL (ALGOrithmic Language) fue un lenguaje de programación que influyó en muchos lenguajes posteriores. Introdujo conceptos como bloques de código estructurados y recursividad. ALGOL 60 fue una versión ampliamente utilizada y sentó las bases para otros lenguajes como Pascal y C.
- **C (1972):** C es un lenguaje de programación de propósito general que fue desarrollado por Dennis Ritchie en los laboratorios Bell. C se convirtió en uno de los lenguajes más populares debido a su eficiencia y su capacidad para acceder directamente al hardware de la computadora. Muchos lenguajes posteriores se basaron en la sintaxis y los conceptos de C.

Tech Hub <FP> euroformac

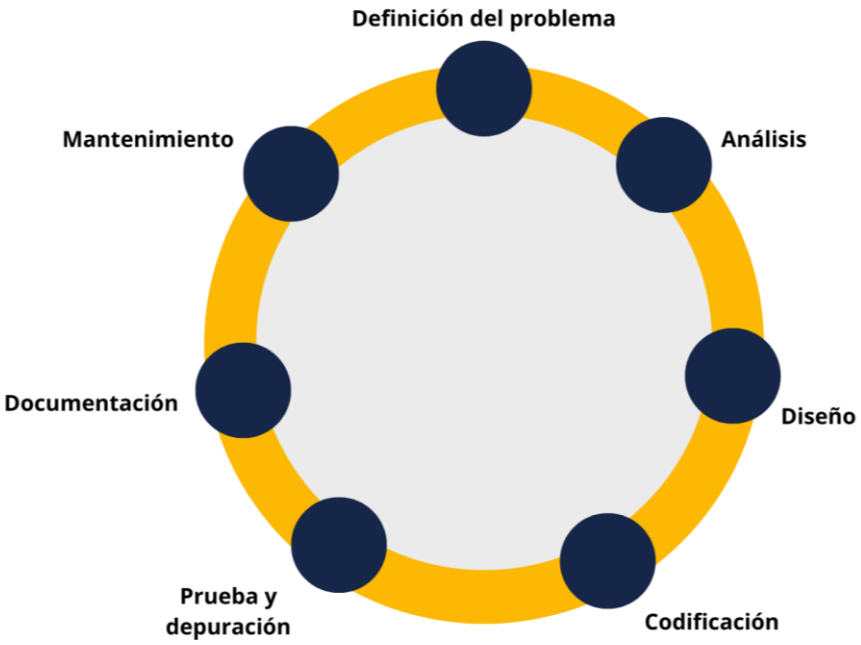
- **Pascal (1970):** Pascal fue desarrollado por Niklaus Wirth como un lenguaje de enseñanza. Pascal enfatizó la claridad y la legibilidad del código y se utilizó ampliamente en entornos académicos. También fue la base para el desarrollo de lenguajes como Ada y Modula-2.
- **C++ (1983):** C++ es una extensión del lenguaje C que añade características de programación orientada a objetos. Fue desarrollado por Bjarne Stroustrup y se convirtió en uno de los lenguajes más populares en el desarrollo de software.

4.4. Ingeniería del Software

Definición

La ingeniería del software debe crearse de manera organizada y secuencial. La programación es sólo una etapa del proceso de desarrollo o ingeniería del software.

Se deben seguir una serie de procedimientos conocidos como Etapas de la ingeniería de software para poder programar grandes aplicaciones con miles de líneas de instrucciones o programas sencillos con pocas instrucciones.



Esquema de Ingeniería del Software

A continuación, describiremos las etapas o pasos de la ingeniería del software.

- **Definición del problema.**

El enunciado claro y complejo del problema. Hay que tener claro "lo que queremos conseguir al final del proceso" porque si no se entiende esto no se puede pasar al siguiente paso.

- **Análisis de datos.**

Dado que es necesaria una definición precisa del problema para encontrar una solución eficiente, las especificaciones de entrada y salida deben describirse en detalle.

Una vez identificado y aclarado el problema, es necesario examinar los siguientes factores:

  - Los resultados esperados.
  - Datos de entrada disponibles.
  - Recursos accesibles (fórmulas, tablas, accesorios varios) que podemos utilizar para manipular los datos y producir un resultado.
- **Diseño de algoritmos.**

El diseño de algoritmos es un proceso fundamental en la programación y la resolución de problemas. Un algoritmo es una serie de pasos o instrucciones bien definidas que resuelven un problema o realizan una tarea específica. Aquí hay algunos pasos generales que podemos seguir al diseñar algoritmos:

  - Comprende el problema: Antes de comenzar a diseñar un algoritmo, es importante comprender completamente el problema que estamos tratando de resolver. Analiza los requisitos, las restricciones y los objetivos del problema. Definir claramente cuál es la entrada (datos de entrada) y cuál debería ser la salida (resultado esperado).
  - Divide y conquista: Si el problema es complejo, puede ser útil dividirlo en subproblemas más pequeños y abordarlos por separado. Dividir el problema en partes más manejables y determina cómo se relacionan entre sí.
  - Diseña una solución general: Una vez que hayamos dividido el problema en subproblemas más pequeños, puedes diseñar una solución general. Pensar en el enfoque general que utilizaremos para resolver el problema y cómo se pueden combinar las soluciones de los subproblemas para obtener la solución final.
- **Codificación del programa.**

Obtención de código fuente y objeto de programas. Conseguir un programa concreto que la máquina pueda entender es a lo que se refiere. Consta de

dos etapas: la creación del programa en lenguaje de alto nivel (código fuente) y su compilación, que consiste en traducir el lenguaje de alto nivel a lenguaje máquina para crear el código objeto del programa.

- **Depurar y probar.**

Tan pronto como obtengamos el programa ejecutable, debemos probarlo. Lo primero que debemos verificar es si aborda adecuadamente el problema en cuestión. Lo someteremos a una variedad de pruebas, cuya naturaleza suele estar determinada por el tipo de problema que se aborda. Una forma común de comenzar a probar un programa es ingresar datos válidos, no válidos e inconsistentes y observar cómo responde cada vez. Localizar errores y corregirlos, si los hay, es el objetivo del proceso de depuración. La depuración se puede considerar como una fase de revisión en la que se modifican los detalles para mejorar el rendimiento del programa si no hay errores.

- **Documentación.**

El manual o guía escrito de un programa es lo que describe los pasos a seguir para usar el programa. El trabajo de un programador es utilizado con frecuencia por otro programador. Como resultado, la documentación ayuda en la comprensión del programa, el uso y la facilitación de posibles modificaciones futuras.

- **Mantenimiento.**

Indica las actualizaciones que se aplicarán al software cuando las circunstancias lo requieran. Este programa debe ser lo suficientemente flexible para adaptarse a los cambios en las circunstancias operativas. La documentación del programa debe reflejar cualquier actualización o modificación.

5. Herramientas y entornos para el desarrollo de programas

5.1 Entornos de desarrollo

Un IDE (Entorno de Desarrollo Integrado) es un programa informático compuesto por una selección de herramientas de programación que facilitan el trabajo del programador y agilizan la creación de programas. Puede aplicarse a uno o más idiomas.

Cada etapa de desarrollo utiliza un conjunto diferente de herramientas para llegar al resultado final. Para cada etapa, podemos tener diferentes IDE.

Componentes del entorno de desarrollo.

- **Editor de código:** Es el componente principal del IDE. Proporciona un área de trabajo donde puedes escribir y editar tu código fuente. Un editor de código generalmente incluye características como resaltado de sintaxis, autocompletado, indentación automática y sugerencias de código para facilitar la escritura y el mantenimiento del código. El compilador. Se encarga de convertir el código fuente escrito en lenguaje de alto nivel a un lenguaje de bajo nivel que la máquina pueda entender y utilizar para su interpretación y ejecución.
- **Un intérprete:** Traduce las instrucciones a medida que se lleva a cabo la instrucción. A pesar de ser más lentos que los compiladores, están basados en intérpretes y, por lo tanto, son independientes de la máquina.
- **Depurador:** Elimina errores del código fuente y los corrige. Le permite pausar el programa en cualquier punto de interrupción para ver cómo se está ejecutando.
- **Constructor de interfaces gráficas de usuario:** Permite simplificar el proceso de creación de interfaces al habilitar la ubicación del control mediante un editor WYSIWYG (del acrónimo en inglés *What You See Is What You*) de arrastrar y soltar.
- **Gestión de versiones:** Permite controlar las revisiones y versiones de las aplicaciones en cualquier momento teniendo control sobre los cambios realizados en las mismas.

Tech Hub <FP> euroformac

Más info

Estas son las páginas web de los entornos de desarrollo más utilizados son los siguientes:

Oracle: http://www.oracle.com/us/downloads/index.html

Eclipse: http://www.eclipse.org

Pycharm: https://www.jetbrains.com/es-es/pycharm/download/#section=windows

5.2 Herramientas de control de versiones

¿Cómo funciona el control de versiones?

El control de versiones en el desarrollo de software es un sistema que registra y gestiona los cambios realizados en el código fuente y otros archivos relacionados a lo largo del tiempo. Su objetivo principal es permitir a los equipos de desarrollo trabajar de manera colaborativa, controlar el historial de cambios, revertir a versiones anteriores, y coordinar eficientemente el desarrollo de software.

A continuación, detallaremos cómo funciona el control de versiones:

- **Repositorio**: El control de versiones utiliza un repositorio centralizado o distribuido para almacenar el código fuente y los archivos del proyecto. En un repositorio centralizado, existe un único servidor que almacena la última versión del proyecto y los desarrolladores trabajan directamente en él. En un repositorio distribuido, cada desarrollador tiene una copia completa del repositorio en su máquina local y los cambios se sincronizan entre las diferentes copias.
- **Commits**: Los cambios en el código se registran mediante "commits". Un commit es una instantánea de los archivos modificados en un momento dado, junto con un mensaje descriptivo que explica los cambios realizados. Los commits suelen ser pequeñas unidades lógicas de cambio y deben estar relacionados con una tarea o función específica.
- **Ramas**: El control de versiones permite la creación de ramas (branches) que son copias independientes del código fuente. Las ramas se utilizan para desarrollar características nuevas o trabajar en correcciones de errores sin afectar la versión principal del proyecto. Una vez que una rama está completa, puede ser fusionada (merge) con la rama principal.
- **Fusiones**: La fusión (merge) es el proceso de combinar los cambios realizados en una rama con otra. Cuando una función o tarea se ha completado en una rama, los cambios se fusionan con la rama principal o con otra rama específica. Esto permite integrar el trabajo de diferentes

desarrolladores y asegurarse de que todos los cambios estén incluidos en la versión final.

- **Conflictos**: En ocasiones, pueden ocurrir conflictos durante la fusión si dos o más desarrolladores han modificado la misma parte de un archivo. En esos casos, el sistema de control de versiones notificará el conflicto y los desarrolladores deben resolverlo manualmente. Esto implica revisar los cambios en conflicto y decidir cómo combinarlos o qué cambios descartar.
- **Historial y versionado:** El control de versiones mantiene un historial completo de todos los cambios realizados en el proyecto, lo que permite acceder a versiones anteriores en cualquier momento. Esto es útil para recuperar versiones anteriores, identificar quién realizó un cambio específico o revertir a una versión estable en caso de problemas.

Algunas herramientas populares de control de versiones son Git, Subversion (SVN) y Mercurial. Estas herramientas proporcionan una interfaz de línea de comandos o una interfaz gráfica para realizar las operaciones mencionadas anteriormente y facilitar la colaboración en el desarrollo de software.

¿Cómo se definen exactamente las versiones?

Usar un número o grupo de números para identificar una versión de un documento es la práctica más típica. Los números de versión se asignan aleatoriamente, no existe un sistema establecido. El uso de números de tres cifras o decimales son dos ejemplos de convenciones de numeración comunes.

Versiones X. Y. Z.

Las versiones en el desarrollo de software se definen utilizando un sistema de numeración que consta de tres componentes principales: la versión principal (x), la versión secundaria (y) y la versión de revisión o parche (z). Esta convención se conoce comúnmente como "versión x.y.z" o "número de versión semántica". A continuación, explicamos el significado de cada componente:

- **Versión principal (x):** El número de versión principal se incrementa cuando se realizan cambios importantes en el software que pueden incluir nuevas características, cambios arquitecturales o modificaciones que pueden romper la compatibilidad con versiones anteriores. Un cambio en la versión principal indica que puede haber cambios significativos y requiere una atención especial durante la actualización.

- **Versión secundaria (y):** El número de versión secundaria se incrementa cuando se agregan nuevas funcionalidades al software,

pero sin afectar la compatibilidad con versiones anteriores. Estos cambios suelen ser incrementales y se consideran evolutivos. Una nueva versión secundaria puede incluir mejoras, adiciones de funcionalidad y correcciones de errores, pero no se espera que rompa la compatibilidad con versiones anteriores.

- **Versión de revisión o parche (z):** El número de versión de revisión o parche se incrementa cuando se realizan correcciones de errores o se solucionan problemas menores en el software. Estas actualizaciones suelen ser pequeñas y no deben introducir cambios significativos en la funcionalidad o la compatibilidad.

El uso de este esquema de numeración permite a los desarrolladores y usuarios del software comprender rápidamente el nivel de cambios y mejoras que se han realizado en una versión determinada. Además, facilita la gestión de las actualizaciones y la comunicación entre los miembros del equipo de desarrollo y los usuarios del software.

Es importante tener en cuenta que el sistema de numeración de versiones puede variar entre diferentes proyectos y organizaciones. Algunas empresas pueden utilizar convenciones diferentes o agregar componentes adicionales, como una versión alfa o beta para indicar un estado de desarrollo temprano o una versión de lanzamiento candidata (release candidate) para indicar una versión final antes del lanzamiento oficial.

¿Qué son exactamente las copias de trabajo?

La copia de trabajo, que se usa para tareas diarias como editar archivos existentes y crear otros nuevos, es una imagen del contenido almacenado en el repositorio de código. Se debe acceder al repositorio antes de que cualquier cambio en esa imagen se considere definitivo. La forma en que se produce esta comunicación entre la copia y el repositorio en funcionamiento tiene un impacto significativo en el control de versiones.

El modelo común de comunicación servidor-cliente es la base de todas las herramientas de control de versiones. Según los documentos, la copia de trabajo está en el cliente, mientras que el repositorio, que es el código, está en el servidor.

Ejemplos de herramientas de control de versiones. GIT, CVS y Apache Subversion (también conocido como SVN) son tres herramientas populares de control de versiones utilizadas en el desarrollo de software. A continuación, proporcionamos una descripción de cada una de ellas:

- **Git:** Git es un sistema de control de versiones distribuido ampliamente utilizado y altamente flexible. Fue desarrollado por Linus Torvalds, el

creador del kernel de Linux. Git utiliza un modelo distribuido en el que cada desarrollador tiene una copia completa del repositorio en su máquina local, lo que permite trabajar de forma independiente y sin conexión a internet. Git es conocido por su rapidez, su capacidad de manejar proyectos grandes y su potente sistema de ramificación y fusión. Además, cuenta con una amplia comunidad de usuarios y es utilizado por proyectos de código abierto y empresas en todo el mundo.

- **CVS (Concurrent Versions System):** CVS es un sistema de control de versiones centralizado que ha sido ampliamente utilizado en el pasado, aunque su popularidad ha disminuido en los últimos años debido al surgimiento de herramientas más modernas como Git. CVS se enfoca en el seguimiento de las versiones de los archivos individuales en lugar de trabajar con el repositorio completo. Permite a varios desarrolladores trabajar en paralelo, pero no tiene la misma capacidad de gestión de ramas y fusiones que Git. CVS aún puede encontrarse en algunos proyectos heredados, pero generalmente se considera una herramienta obsoleta en comparación con las opciones más modernas.

- **Apache Subversion (SVN):** Apache Subversion, también conocido como SVN, es otro sistema de control de versiones centralizado. SVN es considerado como una alternativa más moderna y mejorada a CVS. Proporciona características adicionales, como la capacidad de manejar la copia completa del repositorio en lugar de solo archivos individuales y una mejor administración de versiones. SVN es ampliamente utilizado en proyectos de software propietarios y de código abierto, aunque también ha sido desplazado en gran medida por Git debido a las ventajas que ofrece un sistema distribuido.

Cada una de estas herramientas tiene sus propias características y ventajas, y la elección entre ellas dependerá de las necesidades y preferencias específicas de un proyecto o equipo de desarrollo. Git es ampliamente recomendado y utilizado en la mayoría de los casos debido a su flexibilidad, rendimiento y capacidad de gestión de ramas y fusiones, pero CVS y SVN aún pueden ser utilizados en ciertos escenarios y contextos.

Más info

Aquí podrás acceder a las páginas web de las diferentes herramientas de control de versiones que has aprendido:

https://git-scm.com/download/win

https://ftp.gnu.org/non-gnu/cvs/

https://subversion.apache.org/download.cgi

5.3 Pruebas de software

¿Qué son exactamente las pruebas de software?

Las pruebas de software, también conocidas como garantía de calidad (QA) del software, son un proceso para confirmar y validar la funcionalidad de un programa o aplicación de software para garantizar que el producto de software esté libre de errores. El objetivo final es entregar un producto de alta calidad cumpliendo con los requisitos previstos. Este proceso implica ejecutar software o componentes del sistema a través de herramientas manuales o automatizadas para evaluar una o más propiedades interesantes para el objetivo del software.

Para evitar problemas funcionales, antes de que se lance el producto, la prueba de software es un proceso que se ejecuta simultáneamente con el desarrollo de software.

Las pruebas de software son esenciales para garantizar la calidad, seguridad, confiabilidad y rendimiento de un producto de software. Al detectar problemas tempranamente, se ahorra tiempo y costos, y se brinda una experiencia satisfactoria al cliente. Por lo tanto, es recomendable incluir pruebas de software como parte integral del proceso de desarrollo.

Objetivos de las pruebas de Software

- Encontrar y corregir errores.
- Ofrecer software confiable y de alta calidad.
- Verificar que el producto funcione según lo previsto.
- Evitar volver a cometer los mismos errores.
- Facilitar la toma de decisiones para que la producción solo reciba desarrollos libres de fallos.
- Cumplir los requisitos comerciales y de satisfacción del usuario.
- Evitar futuras ocurrencias de nuevos defectos que afecten el software.

Tipos de pruebas

Las **pruebas funcionales y las pruebas no funcionales** son las dos subcategorías principales de las pruebas de software. Además, existe un tercer tipo de prueba conocido como pruebas de mantenimiento.

- **Pruebas funcionales**: comprueban que un programa o aplicación realiza cada una de las funciones previstas y cumple con un conjunto determinado de requisitos.
- **Pruebas no funcionales o pruebas de rendimiento**: tienen en cuenta factores como la confiabilidad, la usabilidad y el rendimiento.

Mantenimiento (regresión o preservación).

Pruebas de regresión o preservación. Estas pruebas se realizan para asegurarse de que los cambios realizados en el software no hayan afectado las funcionalidades existentes. Se ejecutan los casos de prueba existentes para verificar que el software modificado siga funcionando correctamente y que no se hayan introducido nuevos errores.

5.3.1 Pruebas funcionales

Estas pruebas se centran en verificar si el software cumple con los requisitos y funcionalidades esperadas. El objetivo es comprobar que el software realiza las acciones y procesos específicos de acuerdo con las especificaciones y que produce los resultados correctos. Algunos tipos comunes de pruebas funcionales incluyen:

- **Prueba unitaria:** es una técnica para probar una unidad de código más compacta que se puede aislar lógicamente dentro de un sistema.
- **Pruebas de integración:** examinan la integración o interfaz entre componentes y las interacciones con diferentes partes de un sistema, como el sistema de archivos y el hardware, el sistema operativo o la interfaz entre sistemas.
- **Pruebas del sistema:** miden cómo los diversos componentes de un sistema o aplicación funcionan juntos como un todo.
- **Prueba de sanidad:** es una prueba rápida y fácil para ver si una aplicación o componente específico funciona según lo previsto.
- **Prueba de humo:** es el procedimiento de prueba inicial utilizado para confirmar la funcionalidad fundamental de un programa para asegurarse de que está preparado para el próximo examen.
- **Prueba de interfaz**: confirma la comunicación entre dos sistemas de software diferentes examinando la autenticación de la conexión.

- **Prueba de regresión**: verifica que un programa siga funcionando según lo previsto después de una actualización, modificación o mejora del código.
- **Pruebas de aceptación** (también conocidas como pruebas beta): antes de comercializar o entregar un producto o prototipo, se realizan para determinar si se han cumplido los requisitos del contrato. Garantiza que la funcionalidad, usabilidad, durabilidad y seguridad del producto cumplan con los estándares de calidad y diseño.

5.3.2 Las pruebas no funcionales incluyen

Estas pruebas se enfocan en aspectos no relacionados directamente con la funcionalidad del software, sino en características como el rendimiento, la seguridad, la usabilidad y la escalabilidad. Algunos tipos de pruebas no funcionales incluyen:

- **Las pruebas de rendimiento**, también conocidas como pruebas de optimización del rendimiento, miden la rapidez o la capacidad de respuesta de una aplicación a la carga de trabajo especificada.
- **Prueba de carga:** probar el comportamiento de una aplicación bajo una gran carga de trabajo se conoce como **prueba de carga**.
- **Pruebas de estrés**: las pruebas de estrés evalúan la solidez del software examinando su rendimiento en condiciones de carga extremadamente alta, lo que garantiza que no se bloquee en circunstancias cruciales.
- **Pruebas de seguridad**: determinan si el sistema está protegido contra ataques no anunciados o planificados de fuentes internas y externas.
- **Prueba de instalación**: verifica si una pieza de software funciona según lo previsto después de la instalación.
- **Pruebas de recuperación**: la capacidad de un sistema para recuperarse de errores se prueba durante las **pruebas de recuperación.**
- **Las pruebas de confiabilidad** verifican que el programa funcione de manera consistente al hacer que complete una tarea sin errores dentro de un marco de tiempo predeterminado.
- **Las pruebas de usabilidad:** Si los cambios afectan la interfaz de usuario o la experiencia del usuario, se deben realizar pruebas de usabilidad para verificar que los cambios sean intuitivos, fáciles de usar y satisfagan las necesidades de los usuarios.
- **Pruebas de cumplimiento:** antes de ser lanzado a producción, un programa o sistema de software debe pasar **pruebas de cumplimiento** para demostrar que cumple con un conjunto específico de estándares internos o externos.

- **Prueba de localización:** prueba el comportamiento de un producto en diferentes contextos locales o culturales.

Importante

Una buena implementación de los procesos de testing, ejecutados profesionalmente, disciplinados y sincronizados entre diferentes conjuntos de pruebas, nos evitará que completemos el desarrollo de nuestras aplicaciones con poca o mala calidad, ya que el software puede volverse pobre o incluso desactualizado y dar resultados inesperados.

5.4 Documentación de los programas

El término "documentación" implica que se refiere a todo tipo de información que guía el trabajo del equipo de desarrollo y ayuda a comprender la arquitectura y el diseño de la aplicación a lo largo del tiempo.

La documentación de un programa comienza tan pronto como se construye. Hacer diagramas de los componentes, la funcionalidad, las interfaces, etc. del programa es en realidad lo más típico que se hace primero para que podamos definir claramente qué se desarrollará y cómo lo haremos antes de comenzar a programar.

La preparación de la documentación no finaliza con la entrega de la aplicación porque necesita ser actualizada durante el mantenimiento para reflejar las modificaciones que fueron necesarias para agregar nuevas funcionalidades.

Dentro de un documento de un programa se dará información del proyecto de software sobre los siguientes puntos:

- Indicaremos claramente cómo ejecutar el programa (firma completa del programa, tal como lo haría cualquier sinopsis de una página de manual), enumerando qué parámetros son opcionales y cuáles son necesarios, así como documentando minuciosamente el propósito de cada parámetro y la acción predeterminada que se tomará en la ausencia de cualquier parámetro requerido. Esto, con frecuencia, constituye el manual del usuario final de la aplicación.
- Incluiremos diagramas de flujo y explicaciones a nivel de método de la solución, donde debemos describir la estrategia de resolución general para que las interrelaciones entre los módulos sean claras.
- Incluiremos una sección de "Conclusiones" en la que se presenten los problemas que surgieron durante el desarrollo del proyecto, las

estrategias utilizadas para abordarlos, las limitaciones del problema original, las instancias específicas y, finalmente, los detalles relacionados con la experiencia obtenida a partir de la debe resumirse el tema del proyecto.

Más info

Aplicaciones como el programa Doxygen permiten la generación automática de documentación. Aquí podrás acceder a su página web: http://www.doxygen.nl

Tech Hub <FP> euroformac

6. Errores y calidad de los programas

6.1 Errores

Definición

Un **error de software** es un problema con un programa informático o un sistema de software que conduce a resultados inesperados. Los programas que ayudan a detectar y eliminar errores de software se denominan depuradores.

Cuando hablamos de errores en un lenguaje de programación o su entorno, es importante distinguir entre dos categorías principales: **errores de tipo excepción** y **errores provocados por programadores de aplicaciones** (errores sintácticos o lógicos en el algoritmo del problema). Estos últimos son causados por circunstancias específicas que surgen cuando ocurre un evento imprevisto o inesperado durante el curso de la ejecución de un programa.

Manejo de excepciones:

Se crea un mecanismo de hardware o software denominado **manejo de excepciones** para hacer frente a situaciones en las que surge una condición que modifica la forma en que se ejecutan normalmente los programas. Y esta circunstancia es lo que se denomina una excepción.

En la mayoría de los casos, el estado actual se guarda en un lugar específico y la ejecución del programa continúa a través de lo que normalmente sería un controlador de excepciones.

El programador puede usar los datos de estado guardados para indicarle al programa que reanude la ejecución más tarde desde la posición actual, según las circunstancias. Hay algunas excepciones, como la división por cero, que detienen el programa de inmediato.

En un escenario ideal, nos gustaría que el sistema siguiera funcionando sin sufrir una pérdida grave de funcionalidad. Preferiríamos una degradación gradual, en la que el sistema siga funcionando a una capacidad reducida hasta que pueda reanudar su funcionamiento normal (por ejemplo, en caso de que falle un periférico).

Un error hace que se genere la excepción correspondiente, lo que permite que se maneje la excepción. Se podría pensar en un mecanismo de recuperación directa como el mecanismo de manejo o tratamiento de excepciones: en lugar de volver al estado inicial, el control se transfiere al programador, quien será el encargado de iniciar los procesos de recuperación. Aunque, como veremos en un momento, la recuperación inversa también se puede realizar mediante el manejo de excepciones.

El **manejo de excepciones** debe cumplir con una serie de requisitos generales:

- Debe ser sencillo de comprender y aplicar el tratamiento.
- El código utilizado para manejar la excepción no debe dificultar la comprensión de cómo se comporta normalmente el programa. La combinación de código para el procesamiento de rutinas con código para el manejo de excepciones dará como resultado un programa que generalmente es difícil de entender y mantener, lo que hace que el sistema sea menos confiable.
- El mecanismo debe construirse de modo que los gastos generales de tiempo de ejecución solo ocurran cuando se manejan excepciones y nunca cuando un programa se ejecuta normalmente.
- El mecanismo debe permitir un manejo uniforme de las excepciones, ya sean descubiertas por el entorno o por el programa.
- Las acciones de recuperación se pueden programar utilizando el mecanismo de excepción.

Errores de programación comunes que necesitan el manejo de excepciones:

División por cero
Ciclo infinito
Problemas aritméticos como desbordamientos (overflow) o subdesbordamientos (underflow).
Exceder el tamaño del array definido
Utilizar una variable no inicializada
Errores tipográficos varios, por ejemplo confundir el dígito "0" con la vocal "O", confundir el dígito "1" con la letras "l" o "I", o escribir "," en lugar de "."
Acceder a memoria no permitida (Violación de acceso)

Pérdida de memoria (memory leak)
Desbordamiento o subdesbordamiento de la pila (estructura de datos) (Stack overflow)
Desbordamiento de búfer (buffer overflow)
Bloqueo mutuo (deadlock)
Indizado inadecuado de tablas en bases de datos.
Corrupción de las bases de datos relacionales por no estar normalizadas.
Desbordamiento de la pila de recursión, cuando se dejan demasiadas llamadas en espera.
Programar por ensayo y error, en lugar de desarrollar los programas sistemáticamente.
No atender los mensajes de advertencia al momento de compilar un programa.
Errores que resultan del uso indiscriminado de variables de alcance global.
Errores por no declarar el tipo de las variables.
Errores que emergen porque el programador no toma en cuenta la semántica del lenguaje de programación. Por ejemplo, desconocer si el alcance es léxico o dinámico.
Errores de sintaxis, por ejemplo, no terminar una instrucción con un punto y coma (;) en un lenguaje de programación que lo requiere.
No documentar correctamente el programa, provoca el no entender qué hace, lo que acarrea errores por no tener claro lo que hace.
Condición de carrera (race condition)
Uso incorrecto de API.
Incompatibilidad de sistemas.

6.2 Calidad del software

Definición

La calidad del software se refiere a la medida en que un software cumple con los requisitos establecidos y satisface las necesidades de los usuarios. La calidad del software es crucial, ya que un software de baja calidad puede provocar errores, fallos y problemas de rendimiento, lo que puede tener un impacto negativo en la experiencia del usuario, la productividad y la reputación de la empresa.

Existen diferentes aspectos y características que se consideran al evaluar la calidad del software. Algunos de los factores clave incluyen:

- **Funcionalidad**: El software debe realizar todas las funciones y tareas especificadas correctamente y de manera confiable.
- **Fiabilidad**: El software debe ser confiable y proporcionar resultados consistentes, sin errores ni fallos inesperados.
- **Usabilidad**: El software debe ser fácil de usar, intuitivo y comprensible para los usuarios. Debe contar con una interfaz de usuario clara y bien diseñada.
- **Eficiencia**: El software debe utilizar los recursos de manera eficiente, como el tiempo de procesamiento, la memoria y el almacenamiento. Debe funcionar de manera rápida y sin consumir recursos innecesarios.
- **Mantenibilidad**: El software debe ser fácil de mantener y modificar. Debe contar con un código limpio y bien estructurado, así como con una documentación clara para facilitar las futuras actualizaciones y correcciones.
- **Portabilidad**: El software debe poder ejecutarse en diferentes entornos y plataformas sin problemas.
- **Seguridad**: El software debe ser seguro y proteger los datos confidenciales de los usuarios. Debe contar con medidas de seguridad adecuadas para prevenir accesos no autorizados y proteger contra vulnerabilidades.

Existen estándares y metodologías para evaluar y garantizar la calidad del software, como el modelo de calidad ISO/IEC 25010 (anteriormente conocido como ISO/IEC 9126) y los métodos de desarrollo de software como el enfoque de desarrollo ágil.

Tech Hub <FP> euroformac

La calidad del software se refiere a la medida en que un software cumple con los requisitos establecidos y satisface las necesidades de los usuarios. La calidad del software es crucial, ya que un software de baja calidad puede provocar errores, fallos y problemas de rendimiento, lo que puede tener un impacto negativo en la experiencia del usuario, la productividad y la reputación de la empresa.
En resumen, la calidad del software es esencial para garantizar un producto confiable, eficiente y satisfactorio para los usuarios. Los desarrolladores y las organizaciones deben esforzarse por seguir buenas prácticas de desarrollo, realizar pruebas exhaustivas y adoptar enfoques de mejora continua para asegurar la calidad del software.

Si hablamos de calidad del software, una de las primeras definiciones decía que " la calidad de un programa o sistema se evalúa según el número de defectos por cada mil líneas de código" .

La Organización Internacional de Normalización, ISO en la norma 8402:1994, lo define como la "Totalidad de propiedades y características de un producto, proceso o servicio que le confiere su capacidad para satisfacer necesidades expresas o implícitas".

El control de calidad en el desarrollo de software lo lleva a cabo el desarrollador, que tiene un tiempo limitado, cuando lo tiene. En otros casos, cae bajo un conjunto de patrones rígidos (*checklists* o listas de verificación en inglés) cuya revisión suele llevar demasiado tiempo. También puede estar relacionado con las etapas del ciclo de vida del desarrollo de software.

Comenzando con las acciones tomadas durante la planificación, el aseguramiento de la calidad incluye actividades de auditoría como revisiones o inspecciones técnicas, maximizando los criterios de corrección e integridad, y funciones de gestión de la información que están más enfocadas en el desarrollo y la documentación de las pruebas.

El grado en que las cinco fases del ciclo de vida del desarrollo de software (análisis, diseño, implementación, prueba e implementación) y cada subproceso o fase o etapa del desarrollo de software se llevan a cabo con calidad determinará la calidad general del proceso. La claridad, la implicación, la planificación, los estándares, la formación, la experiencia, los controles, la documentación, el soporte y la finalización son otros elementos cruciales que deben estar presentes en cualquier proyecto. Debemos destacar que las actividades organizativas, técnicas y humanas (formación y motivación del personal) se realizan como parte del proceso de mejora continua. Las actividades técnicas incluyen la aplicación de la ingeniería de software.

Debido a las características únicas del software, como producto y como servicio, es necesario adoptar un estándar de calidad basado en el conocimiento de otras industrias más maduras. En esta unidad hacemos referencia al Capability Maturity Model Integration, también conocido como CMMI, el cual es un modelo enfocado a la mejora creado por la Universidad Carnegie Mellon (EE. UU.) en el Software Engineering Institute - SEI (por sus siglas en inglés Software Engineering Institute) de procedimientos en varios niveles de madurez empresarial.

6.3 Estándares de calidad del software

ISO/IEC 25010

El modelo de calidad es la base del sistema de evaluación de la calidad del producto. En este modelo, las características de calidad se tendrán en cuenta al evaluar los atributos de un producto de software en particular que se seleccionará ("ISO 25010").

La calidad de un producto de software puede entenderse como la medida en que el producto satisface las necesidades del usuario y, por lo tanto, proporciona valor. Precisamente estos requisitos (función, rendimiento, seguridad, mantenimiento, etc.).

El modelo de calidad del producto definido por la norma ISO/IEC 25010 proporciona un marco de referencia para evaluar la calidad del software y otros productos de tecnología de la información. Este modelo se basa en ocho características principales de calidad, divididas en dos categorías: características de calidad interna y características de calidad externa. A su vez, estas características se desglosan en subccaracterísticas más específicas. A continuación, se detallan las ocho características principales de calidad del producto según la norma **ISO/IEC 25010**.

Calidad del producto software	**Adecuación Funcional**	- Completitud funcional - Corrección funcional - Pertinencia funcional
	Eficiencia de desempeño	- Comportamiento temporal - Utilización de recursos - Capacidad
	Compatibilidad	- Coexistencia - Interoperabilidad

	Usabilidad	- Reconocibilidad de la adecuación - Aprendizabilidad - Operabilidad - Protección frente a errores de usuario - Estética - Accesibilidad
	Fiabilidad	- Madurez - Disponibilidad - Tolerancia a fallos - Capacidad de recuperación
	Seguridad	- Confidencialidad - Integridad - No repudio - Autenticidad - Responsabilidad
	Mantenibilidad	- Modularidad - Reusabilidad - Analizabilidad - Capacidad de ser modificado - Capacidad de ser probado
	Portabilidad	- Adaptabilidad - Facilidad de instalación - Capacidad de ser reemplazado

Calidad del software

Adecuación Funcional

Representa la capacidad de un producto de software para proporcionar una funcionalidad que satisfaga necesidades específicas e implícitas, cuando el producto se utiliza en condiciones específicas. Esta función se divide en las siguientes subfunciones:

- **Completitud funcional.** La medida en que el conjunto de funciones incluye todas las actividades definidas y los objetivos del usuario.
- **Corrección funcional.** La capacidad de un producto o sistema para lograr resultados precisos con el grado de precisión requerido.

- **Pertinencia funcional.** La capacidad de un producto de software para proporcionar un conjunto adecuado de funciones para las tareas y objetivos específicos del usuario.

Eficiencia de desempeño

Esta propiedad representa el rendimiento en relación con la cantidad de recursos utilizados en determinadas condiciones. Esta función se divide en las siguientes subfunciones:

- **Comportamiento temporal.** Los tiempos de respuesta, el procesamiento y las tasas de rendimiento de un sistema cuando realiza sus funciones bajo ciertas condiciones relacionadas con un banco de prueba específico (*benchmark*).
- **Utilización de recursos.** El número y tipo de recursos usados cuando el programa realiza su función bajo condiciones específicas.
- **Capacidad.** La medida en que los límites superiores de un parámetro de producto o sistema de software cumplen los requisitos.

Compatibilidad

El software debe ser compatible con otros sistemas, componentes o productos con los que necesite interactuar, como sistemas operativos, navegadores web o bases de datos. Esta función se divide en las siguientes subfunciones:

- **Coexistencia.** La coexistencia se refiere a la capacidad de múltiples sistemas o productos para funcionar simultáneamente en un entorno compartido sin interferencias ni conflictos.

- **Interoperabilidad.** La interoperabilidad se refiere a la capacidad de diferentes sistemas, productos o componentes para comunicarse, compartir información y trabajar juntos de manera efectiva.

Usabilidad

La capacidad del producto de software para ser entendido, aprendido, usado y atractivo para los usuarios, cuando se usa bajo ciertas condiciones. Esta función se divide en las siguientes subfunciones:

- **Reconocibilidad de la adecuación.** La funcionalidad del producto permite a los usuarios comprender si el software se ajusta a sus necesidades.
- **Aprendizabilidad.** La funcionalidad del producto permite a los usuarios conocer sus aplicaciones.

- **Operabilidad.** Las capacidades del producto permiten al usuario operarlo y controlarlo con facilidad.
- **Protección contra errores de usuario.** La capacidad del sistema para proteger a los usuarios de cometer errores.
- **Estética de la interfaz de usuario.** La capacidad de la interfaz de usuario para satisfacer y satisfacer la interacción del usuario.
- **Accesibilidad.** La capacidad del producto para permitir su uso por parte de usuarios con determinadas características y discapacidades.

Fiabilidad

La capacidad de un sistema o componente para realizar funciones específicas, cuando se utiliza en condiciones específicas y durante un período de tiempo específico. Esta función se divide en las siguientes subfunciones:

- **Madurez.** La capacidad del sistema para satisfacer las necesidades de confiabilidad en condiciones normales.

- **Disponibilidad.** La capacidad de un sistema o componente para funcionar y es accesible para su uso cuando sea necesario.

- **Tolerancia a fallos.** La capacidad de un sistema o componente para funcionar según lo previsto en el caso de un fallo de hardware o software.

- **Capacidad de recuperación.** La capacidad de recuperación del software se refiere a su capacidad para recuperarse de fallos, errores o interrupciones, y restablecer su funcionamiento normal de manera rápida y confiable. Es un aspecto importante de la calidad del software, ya que los fallos pueden ocurrir debido a diversos factores, como errores de programación, problemas de hardware, errores del usuario o eventos inesperados.

Seguridad

La capacidad de seguridad del software se refiere a su capacidad para proteger los datos, sistemas y recursos contra amenazas y ataques maliciosos. La seguridad del software es fundamental para garantizar la confidencialidad, integridad y disponibilidad de la información, así como para prevenir el acceso no autorizado, el robo de datos y otros riesgos de seguridad. Esta función se divide en las siguientes subfunciones:

- **Confidencialidad.** La capacidad de proteger contra el acceso no autorizado a datos e información, accidental o intencional.

- **Integridad.** La capacidad de un sistema o componente para evitar el acceso no autorizado o la modificación de datos o programas informáticos.
- **No repudio.** La capacidad de probar que las acciones o eventos ocurrieron de tal manera que esas acciones o eventos no pueden negarse.
- **Responsabilidad.** La capacidad de realizar un seguimiento explícito de las acciones de la entidad.

- **Autenticidad.** La capacidad de probar la identidad de un individuo o proveedor.

Mantenibilidad

Capacidad para modificar de manera eficiente y efectiva el producto de software para satisfacer las necesidades de evolución, corrección o mejora. Esta función se divide en las siguientes subfunciones:

- **Modularidad.** La capacidad de un sistema o programa informático (que consta de componentes discretos) para permitir que se realicen cambios en un componente con un impacto mínimo en otros componentes.
- **Reusabilidad.** La capacidad de un activo le permite ser utilizado en más de un sistema de software o para crear otros activos.
- **Documentación adecuada:** Es importante contar con documentación clara y completa que describa la arquitectura, el diseño, las funcionalidades y las dependencias del software. Esto ayuda a los desarrolladores a comprender rápidamente el sistema y facilita las tareas de mantenimiento y modificación.
- **Gestión del cambio:** Es importante tener procesos y herramientas adecuados para gestionar y controlar los cambios en el software. Esto implica tener un sistema de control de versiones, seguimiento de problemas y un flujo de trabajo definido para evaluar, aprobar e implementar los cambios en el sistema.
- **Capacidad para ser probado.** El software debe ser diseñado de manera que las pruebas de regresión y las pruebas unitarias puedan realizarse de manera eficiente. Esto permite detectar y corregir errores de manera más rápida y garantiza la estabilidad del sistema después de las modificaciones.

Portabilidad

La portabilidad del software se refiere a su capacidad para ser ejecutado o utilizado en diferentes entornos o plataformas sin necesidad de realizar modificaciones significativas. Es decir, un software portátil puede ser fácilmente trasladado o instalado en diferentes sistemas operativos, dispositivos o arquitecturas sin perder funcionalidad ni requerir cambios extensos en el código.

Algunos aspectos clave relacionados con la portabilidad del software son:

- Independencia del sistema operativo: Un software portátil está diseñado para ser compatible con diferentes sistemas operativos, como Windows, macOS, Linux, iOS o Android. Esto implica utilizar estándares y tecnologías comunes que sean compatibles en múltiples plataformas, como lenguajes de programación multiplataforma (por ejemplo, Java), bibliotecas y frameworks que sean interoperables.
- Interoperabilidad: El software portátil debe ser capaz de interactuar y comunicarse con otros sistemas, aplicaciones o servicios, independientemente de la plataforma en la que se ejecuten. Esto implica utilizar estándares y protocolos abiertos que sean ampliamente aceptados y soportados en diferentes entornos.
- Abstracción de hardware: El software portátil debe ser independiente de la arquitectura de hardware subyacente. Esto se logra mediante el uso de abstracciones y capas de software que ocultan las características específicas del hardware y proporcionan una interfaz común para el software.
- Gestión de dependencias: El software portátil debe gestionar correctamente las dependencias de otros componentes o bibliotecas que pueda requerir para su funcionamiento. Es importante asegurarse de que todas las dependencias necesarias estén disponibles y sean compatibles en el entorno objetivo.
- Pruebas y validación en diferentes entornos: Es fundamental realizar pruebas exhaustivas del software en diferentes entornos y plataformas objetivo para garantizar su correcto funcionamiento y detectar posibles problemas de compatibilidad.
- La portabilidad del software es importante para maximizar su alcance y facilitar su distribución y adopción en diferentes entornos. Al desarrollar software portátil, se promueve la reutilización de código, se reducen los costos y esfuerzos asociados con el desarrollo de versiones específicas para cada plataforma, y se ofrece una mayor flexibilidad para los usuarios en términos de elección de plataforma y dispositivos.

6.4 Conclusiones

En este apartado, hemos visto algunas definiciones básicas de calidad y cómo se relaciona con el software, todo lo cual se ha simplificado. Esto evoca la idea de cómo la necesidad de diferenciación de la industria ha evolucionado con el tiempo y cómo esta necesidad ha influido en la creación de estándares de calidad. La aplicación es necesaria dado el estado actual de la industria del software.

Tech Hub <FP> euroformac

7. Resumen

En esta primera unidad se presentan los conceptos necesarios para seamos capaces de entender las sucesivas unidades, nos familiaricemos con las herramientas necesarias y seamos capaces de entender las metodologías de trabajo empleadas por los equipos de desarrolladores.

Es muy **importante profundizar en los conceptos de los apartados 1 y 2** y entender haciendo varias lecturas los apartados restantes.

Aquí hay un resumen de los temas más importantes tratados en esta unidad:

- Se han visto los conceptos básicos de programación: Introducción a los conceptos fundamentales de la programación, como variables, tipos de datos, operadores y expresiones.
- Hemos podido conocer los paradigmas de programación centrando la atención en la Declarativa e imperativa.
- Otro aspecto importante a la hora de abordar la programación son los tipos de lenguajes de programación que hemos descrito en esta unidad.
- Finalmente, has podido conocer cuáles son los principales errores que puedes cometer y la importancia que tiene la calidad del software.

Tech Hub <FP> euroformac

Glosario

- **Benchmark:** La palabra inglesa Benchmark se usa a menudo para definir comparaciones de desempeño, con el objetivo de determinar qué iniciativa, empresa, política o sistema exhibe el comportamiento más apropiado para un propósito particular.
- **Bucle:** Un bucle en programación es una estructura que permite repetir un bloque de código múltiples veces. Los bucles son una forma eficiente de automatizar tareas repetitivas y evitar tener que escribir el mismo código una y otra vez.
- **Código:** es un conjunto de instrucciones escritas en un lenguaje de programación específico, que es comprensible para los programadores y desarrolladores. El código fuente es el nivel más alto de abstracción en el desarrollo de software y representa la forma en que los programadores expresan sus algoritmos y lógica para que sean entendidos por las computadoras.
- **Matriz:** En programación, una matriz es una estructura de datos que almacena una colección ordenada de elementos del mismo tipo. También se le conoce como array. Una matriz se representa como una tabla rectangular compuesta por filas y columnas.
- **Throughput:** es la cantidad de trabajo o información neta que fluye a través de un sistema, como una red informática.
- **Vector:** Es una matriz de una sola dimensión.

Tech Hub <FP> euroformac

Bibliografía

Allen B. Tucker .2021. “ Lenguajes de programación” , Editorial McGraw Hill.

El tutorial de Python
https://docs.python.org/es/3/tutorial/

Norma ISO 25000.
https://iso25000.com/index.php/normas-iso-25000/iso-25010

Miriam Martínez Canelo, septiembre 2021. Testing de Software:
https://profile.es/blog/que-es-el-testing-de-software/

Tech Hub <FP> euroformac